

DEBRECENI EGYETEM INFORMATIKAI KAR

SZAKDOLGOZAT

Gamifikált fitness mobil alkalmazás

Témavezető:
Dr. Biró Piroska
adjunktus

Készítette:
Abari Gergő
programtervező
informatikus

Debrecen
2026

Tartalomjegyzék

1. Bevezetés.....	3
2. A gamifikáció szerepe mobilalkalmazásokban.....	5
2.1. A gamifikáció fogalma és alapjai.....	5
2.2. A gamifikáció játékelemei és mechanizmusai.....	6
2.3. A gamifikáció hatásossága a különböző alkalmazási területeken.....	6
2.4. A gamifikáció korlátai és kritikái.....	7
2.5. Összegzés.....	8
3. A fejlesztői környezet és technológiák.....	10
3.1. A fejlesztői környezet áttekintése.....	10
3.2. Backend fejlesztői környezet – PhpStorm.....	10
3.3. PHP programozási nyelv.....	11
3.4. Laravel keretrendszer.....	11
3.5. Hitelesítés és biztonság.....	12
3.6. Adatbázis – MySQL.....	13
3.7. Frontend fejlesztői környezet – Android Studio.....	14
3.8. Kotlin programozási nyelv.....	15
3.9. Jetpack Compose.....	15
3.10. Hálózati kommunikáció.....	16
3.11. A rendszer architektúrájának összefoglalása.....	17
4. A LvlUp! Mobilalkalmazás bemutatása.....	19
4.1. Az alkalmazás felépítése és navigációja.....	19
4.2. Felhasználókezelés és hitelesítés.....	21
4.3. Főképernyő.....	25
4.4. Aktivitás kezelése.....	29
4.5. Táplálkozás kezelése.....	34
4.6. Heti statisztika.....	40
4.7. Ranglista rendszer.....	43
4.8. Napi kihívások.....	47
4.9. Továbbfejlesztési lehetőségek.....	50
Irodalomjegyzék.....	51

1. Bevezetés

Napjainkban az egészséges életmód iránti érdeklődés folyamatosan növekszik. Számos kezdeményezés irányul arra, hogy valamilyen módon ösztönözzön másokat, hogy úgy éljék életüket, hogy fizikailag a legtöbbet hozzák ki magukból. A modern társadalom életvitele azonban számos olyan tényezőt hordoz magában, amelyek megnehezítik ennek gyakorlati megvalósítását. Az ülő életmód elterjedése, a rendszertelen táplálkozás, valamint a mindennapi stressz hozzájárulnak ahhoz, hogy az emberek nehezen alakítsanak ki hosszú távon fenntartható, egészségtudatos szokásokat. Az ülő életmód egészségügyi kockázata globális szinten is jelentős: egy 168 országra kiterjedő vizsgálat szerint a fizikai inaktivitás a teljes halálozás mintegy 7,2%-ához járul hozzá [1]. Bár az egészséges életmóddal kapcsolatos információk széles körben elérhetők, ezek alkalmazása a mindennapi gyakorlatban gyakran elmarad.

Az egészséges életmód kialakításának és fenntartásának egyik legnagyobb akadály a motiváció hiánya. Sok esetben a kezdeti lelkesedés gyorsan alábbhagy, és az egyének visszatérnek korábbi szokásaikhoz. Bár számos módszer létezik egy ideális életmód támogatására, például edzéstervek és étrendek követése, vagy különböző motiváló alkalmazások, ezek gyakran nem biztosítanak elegendő visszajelzést vagy élményt a felhasználók számára. Ennek következtében a felhasználók érdeklődése csökken, és a kívánt változás nem válik tartóssá. A szakirodalom rámutat arra, hogy a motiváció hiánya mögött több tényező is állhat, például az a percepció, hogy a viselkedésváltozáshoz szükséges erőfeszítés meghaladja annak várható hasznát [2]. Emellett az egyének sokszor nem képesek tudatosan felülmúlni a korábban kialakult, berögződött szokásaikat, ami tovább nehezíti a változást [2]. A probléma megoldásához olyan módszerre van szükség, amely nemcsak támogatja a változás megkezdését, hanem annak hosszú távú fenntartásához is hozzájárul.

A fenti problémák kezelésére egyre nagyobb figyelem irányul a gamifikáció alkalmazására, amely játékos elemek beépítését jelenti nem játékos környezetben [3]. A gamifikáció célja a felhasználók motivációjának növelése, valamint a kívánt viselkedések kialakításának és fenntartásának támogatása. Az egészségfejlesztés területén ez kifejezetten fontos, mivel a felhasználói elköteleződés kulcsfontosságú a hosszú távú célok elérésében. Egy átfogó

elemzés rámutat arra, hogy az egészségügyi alkalmazásokban leggyakrabban alkalmazott technikák közé tartozik a visszajelzés és önmonitorozás, a célkitűzés, valamint a jutalmazás [4]. Ezek az elemek lehetővé teszik a felhasználók számára, hogy folyamatos visszacsatolást kapjanak saját teljesítményükről, nyomon kövessék fejlődésüket, valamint motivációt nyerjenek a kitűzött célok eléréséhez. A gamifikáció alkalmazása olyan eszközt kínál, amely képes összekapcsolni a viselkedésváltozás elméleti alapjait a gyakorlati megvalósítással, ezáltal hatékonyan támogatja az egészségtudatos életmód kialakítását.

A szakdolgozatom keretében egy ilyen gamifikációs eszközökkel bővített, egészséges életmódra ösztönző mobilalkalmazás megtervezése és fejlesztése valósul meg. Az alkalmazás lehetőséget biztosít a napi aktivitás, a táplálkozás, valamint egyéb egészséggel kapcsolatos tényezők nyomon követésére. A rendszer központi eleme a felhasználók motiválása különböző játékos megoldások segítségével, mint például pontgyűjtés, kihívások teljesítése, szintlépések és statisztikai visszajelzések. A felhasználók ezáltal nemcsak rögzíthetik adataikat, hanem folyamatos visszajelzést is kapnak fejlődésükről, ami hozzájárulhat a hosszú távú elköteleződés fenntartásához. A cél egy olyan rendszer létrehozása, amely egyszerre informatív és motiváló, ezáltal hatékonyan támogatja az egészségtudatos viselkedés kialakulását.

A dolgozat elkészítése során többféle módszer kerül alkalmazásra. Az első lépés a téma elméleti megalapozása, releváns szakirodalmak feldolgozásával. Ezt követően sor kerül a hasonló célú mobilalkalmazások elemzésére, különös tekintettel azok funkcionalitására és motivációs eszközeire. A tervezési szakaszban meghatározásra kerülnek az alkalmazás fő funkciói, valamint kialakításra kerül a rendszer felépítése és felhasználói felülete. A fejlesztési fázis során megvalósításra kerül az alkalmazás működése, majd ezt követően a rendszer tesztelése történik meg, amely elsősorban a működés ellenőrzésére és a stabilitás vizsgálatára irányul.

2. A gamifikáció szerepe mobilalkalmazásokban

2.1. A gamifikáció fogalma és alapjai

A digitális technológiák fejlődésével párhuzamosan egyre nagyobb hangsúlyt kapnak azok a megoldások, amelyek a felhasználói élmény növelésével próbálják elősegíteni a motivációt és a tartós elköteleződést. Különösen a mobilalkalmazások területén figyelhető meg az a tendencia, hogy a pusztán funkcionális rendszerek helyett olyan interaktív megoldások kerülnek előtérbe, amelyek hosszabb távon képesek fenntartani a felhasználó érdeklődését. Ebben a kontextusban vált kiemelt jelentőségűvé a gamifikáció, amely a játékokból ismert elemek és mechanizmusok alkalmazásán keresztül próbálja támogatni a felhasználói aktivitást.

A gamifikáció fogalmának meghatározására a szakirodalomban több megközelítés is létezik, azonban az egyik legszélesebb körben elfogadott definíció Deterding és munkatársai nevéhez köthető. Meghatározásuk szerint a gamifikáció „a játéktervezési elemek alkalmazása nem játékos környezetben” [5]. Ez az értelmezés egyértelműen elkülöníti a gamifikációt a teljes értékű játékoktól, hangsúlyozva, hogy itt nem önálló játékok létrehozásáról van szó, hanem a játékokból ismert struktúrák, például pontozási rendszerek, szintek, kihívások vagy visszacsatolási mechanizmusok alkalmazásáról.

A definícióból következik, hogy a gamifikáció nem azonos a teljes értékű játékokkal, hanem azok elemeinek célszerű alkalmazását jelenti. A hangsúly tehát nem a szórakoztatáson, hanem a felhasználói viselkedés és motiváció befolyásolásán van. A gamifikáció elsődleges célja jellemzően a felhasználói aktivitás növelése és az elköteleződés fenntartása, különösen olyan területeken, ahol a feladatok önmagukban kevésbé motiválóak.

Fontos ugyanakkor, hogy a gamifikáció nem merül ki egyszerű játékelemek alkalmazásában, hanem tudatos tervezést igényel. Ennek megfelelően a következőkben érdemes részletesebben megvizsgálni a gamifikáció során alkalmazott játékelemeket és mechanizmusokat.

2.2. A gamifikáció játékelemei és mechanizmusai

A kutatások alapján számos különböző játékelem azonosítható, azonban ezek közül néhány kiemelkedően gyakran fordul elő. Ilyenek például a pontok, jelvények és ranglisták, amelyek a legtöbb gamifikált rendszer alapját képezik [6]. Ezek az elemek elsősorban jutalmazási és visszacsatolási funkciót töltenek be, lehetővé téve a felhasználó számára az előrehaladás nyomon követését és teljesítménye értékelését.

A játékelemek ugyanakkor nem csupán önálló elemekként értelmezhetők, hanem olyan strukturális építőkövekként is, amelyek hozzájárulnak a rendszer működésének átláthatóságához és a felhasználói élmény kialakításához. Segítségükkel meghatározhatók a célok, a szabályok, valamint az előrehaladás módjai, ezáltal támogatva a felhasználói bevonódást és a motiváció fenntartását [6].

A gyakorlatban ezek az elemek különböző módon jelennek meg a mobilalkalmazásokban. Például egy fitnessalkalmazás pontokat adhat a napi lépésszám vagy edzések teljesítéséért, vagy akár az étrend betartásáért, míg jelvényekkel jutalmazhatja a felhasználót bizonyos mérföldkövek elérésekor, például egy heti cél teljesítése esetén. A ranglisták lehetőséget biztosítanak a felhasználók közötti összehasonlításra, ami versengést idézhet elő, és tovább növelheti az aktivitást. Emellett gyakran alkalmaznak szinteket és kihívásokat is, amelyek fejlődési utat kínálnak, valamint rövid távú célok kitűzésével segítik a felhasználók folyamatos bevonását.

Összességében elmondható, hogy a játékelemek megfelelő kombinációja kulcsszerepet játszik a gamifikáció sikerességében. A jól megtervezett rendszerek nem csupán rövid távú aktivitást eredményezhetnek, hanem képesek hosszabb távon is fenntartani a felhasználók motivációját és elköteleződését.

2.3. A gamifikáció hatásossága a különböző alkalmazási területeken

A gamifikáció hatásosságát az elmúlt években számos kutatás vizsgálta különböző alkalmazási területeken. Az eredmények összességében azt mutatják, hogy a gamifikáció képes növelni a felhasználói aktivitást, motivációt és elköteleződést, ugyanakkor a hatások nem minden esetben egyértelműek. Egy átfogó szakirodalmi áttekintés, amely több száz tanulmány eredményeit elemezte, arra a következtetésre jutott, hogy bár a gamifikáció hatásai

többnyire pozitív irányba mutatnak, jelentős arányban jelennek meg vegyes vagy korlátozott eredmények is [7].

A különböző alkalmazási területek közül kiemelhető az oktatás és az egészségügyi szolgáltatások. Az oktatásban a gamifikáció alkalmazásának egyik fő célja a tanuló bevonásának növelése és a tanulási folyamat élményszerűbbé tétele. A játékos elemek hozzájárulhatnak ahhoz, hogy a tanulók aktívabban vegyenek részt a feladatokban, azonban a tanulmányi teljesítményre gyakorolt hatás sok esetben csak mérsékelt mértékű [8].

Az egészségügyi és életmód-támogató alkalmazások esetében a gamifikáció főként a viselkedésváltozás segítésében jelenik meg. A játékos elemek, mint a jutalmak, szintek vagy visszajelzések, támogathatják a felhasználókat abban, hogy rendszeresen végezzenek egészségmegőrző tevékenységeket, például növeljék fizikai aktivitásukat. A kutatások ezen a területen is többnyire pozitív hatásokat mutatnak, különösen a viselkedés szintjén, ugyanakkor gyakoriak a vegyes eredmények is. Ez arra utal, hogy a gamifikáció hatásossága nagyban függ az alkalmazás módjától és a célcsoporttól [9].

Összességében elmondható, hogy a gamifikáció hatásossága kontextusfüggő. Bár a kutatások többsége pozitív hatásokról számol be, különösen a felhasználói aktivitás és bevonódás terén, az eredmények nem minden esetben egyértelműek [7]. A hatás mértékét jelentősen befolyásolja az alkalmazott játékelemek típusa, azok megvalósítása, valamint a célcsoport jellemzői. Ennek alapján a gamifikáció nem tekinthető univerzálisan hatékony megoldásnak, hanem olyan eszköznek, amely megfelelő tervezés mellett képes támogatni a motivációt és a viselkedésváltozást.

2.4. A gamifikáció korlátai és kritikái

Az eddigiekben a gamifikáció pozitív oldalát vizsgáltuk, azonban mint minden megközelítésnek, ennek is vannak korlátai és kedvezőtlen hatásai. Bár a gamifikációt gyakran a felhasználói motiváció növelésének és az elköteleződés erősítésének hatékony eszközeként tartják számon, a kutatások rámutatnak arra, hogy alkalmazása nem minden esetben vezet a kívánt eredményekhez [10]. Bizonyos helyzetekben kifejezetten negatív következmények is megjelenhetnek, amelyek sokszor nem szándékoltak, és csak a rendszer használata során válnak láthatóvá. Ezért a gamifikáció értékelésekor nem elegendő kizárólag a pozitív

hatásokra koncentrálni, hanem szükséges figyelembe venni annak korlátait és lehetséges kockázatait is.

A kutatások több konkrét negatív hatást is azonosítottak a gamifikáció alkalmazásával kapcsolatban. Gyakran jelentkezik motivációs problémák, például a kezdeti érdeklődés csökkenése vagy a külső jutalmak túlhangsúlyozása. Emellett a játékos elemek elterelhetik a figyelmet a tanulási vagy feladatvégzési célról, ami a megértés romlásához vezethet. További gyakori probléma a rendszer kijátszása, amikor a felhasználók nem a kívánt viselkedést követik, hanem a jutalmazási mechanizmusok kihasználására törekednek [10].

A gamifikáció negatív hatásai között kiemelt szerepet kap a felhasználók közötti folyamatos összehasonlítás is. A különböző ranglisták, és megosztott teljesítményadatok ugyan ösztönözhetik az aktivitást, ugyanakkor negatív pszichológiai következményekkel is járhatnak. A kutatások szerint a másokhoz való viszonyítás negatív önértékelést, bizonytalanságot, frusztrációt és önbizalomcsökkenést okozhat, különösen akkor, ha a felhasználó nála jobban teljesítő személyekhez méri magát [11]. Ez a hatás még erősebben jelentkezhet érzékenyebb adatok esetén, például testzsírszázalékhoz vagy más testi jellemzőkhöz kapcsolódó összehasonlításoknál, amelyek szégyenérzetet vagy fokozott önkritikát is kiválthatnak [11]. Mindez arra utal, hogy a gamifikáció nemcsak motiváló, hanem pszichésen terhelő mechanizmusokat is működésbe hozhat, és a folyamatos megfelelési kényszer nemcsak az összehasonlító funkciókban, hanem más elemeknél, például a napi sorozatok fenntartására épülő rendszerekben is megjelenhet.

Összességében a gamifikáció hatékony motivációs eszköz lehet, azonban fontos a negatív pszichológiai hatások minimalizálása. A versengés és az összehasonlítás egyes felhasználóknál stresszt és önértékelési problémákat okozhat, ezért célszerű olyan rendszereket kialakítani, amelyek az egyéni fejlődést is támogatják. Lényeges, hogy a felhasználók választhassanak: részt kívánnak-e versenyben, vagy inkább saját céljaikra és előrehaladásukra fókuszálnak.

2.5. Összegzés

Összességében a gamifikáció jól használható eszköz a motiváció növelésére és a felhasználók bevonására, de nem minden esetben működik pozitívan. A versengés és az

állandó összehasonlítás könnyen vezethet stresszhez vagy negatív önértékeléshez, különösen érzékenyebb területeken. Ezért fontos, hogy a gamifikációt ne csak ösztönzésre használjuk, hanem úgy alakítsuk ki, hogy az egyéni fejlődést is támogassa. Így megőrizhető az előnye, miközben a kedvezőtlen hatások csökkenthetők.

3. A fejlesztői környezet és technológiák

3.1. A fejlesztői környezet áttekintése

Az alkalmazás két önálló, de szorosan együttműködő komponensre épül: egy szerveroldali backendre és egy Android-alapú mobilalkalmazásra. A két rész eltérő fejlesztői környezetben és technológiai eszközkészlet felhasználásával készült, kommunikációjuk REST-elvű JSON API-n keresztül történik. Ez az architektúra lehetővé teszi, hogy a backend és a frontend egymástól függetlenül fejleszthető és karbantartható legyen.

A szerveroldali komponens fejlesztése a JetBrains PhpStorm integrált fejlesztői környezetben zajlott. A backend PHP programozási nyelven, a Laravel 12 keretrendszer felhasználásával készült, az adatok tárolását MySQL relációs adatbázis-kezelő rendszer biztosítja. A hitelesítés egyedi Bearer token alapon valósul meg, amelyet egy saját réteg ellenőriz.

A mobilalkalmazás fejlesztése a Google Android Studio környezetben zajlott. A frontend Kotlin programozási nyelven íródott, a felhasználói felület kialakítása a Jetpack Compose UI-keretrendszer segítségével történt. A háttérrendszerrel való kommunikációért a Retrofit2 HTTP-kliens könyvtár és az OkHttp hálózati réteg felelős. A projekt build-konfigurációja Kotlin DSL alapú. A célplatform minimális API-szintje 24 a cél API-szint pedig 36.

3.2. Backend fejlesztői környezet – PhpStorm

A backend fejlesztéséhez használt fejlesztői környezet kiválasztásakor fontos szempont volt, hogy az eszköz megfelelő támogatást nyújtson mind a PHP nyelvhez, mind a Laravel keretrendszerhez. Erre a célra a JetBrains PhpStorm IDE bizonyult a legalkalmasabb megoldásnak, amely napjainkban a PHP-alapú webfejlesztés egyik legelterjedtebb professzionális eszköze.

A PhpStorm a JetBrains cég terméke, amelyet kifejezetten PHP fejlesztők számára terveztek. Az IDE legjelentősebb funkciója az intelligens kódkiegészítés, amely nemcsak az alapvető PHP szintaxist ismeri, hanem mélyen integrálódik a projekt szerkezetébe is: osztályneveket, metódusokat, változókat és egész kódsorokat képes kontextusnak megfelelően javasolni. Ez különösen hasznos összetett, keretrendszerre épülő projekteknél, ahol a kézi kódírás lassú és

hibalehetőségekkel teli lenne. Emellett az IDE valós idejű hibakeresési és statikus kódelemzési funkciókkal is rendelkezik, amelyek segítségével a fejlesztés során felmerülő problémák korán azonosíthatók.

3.3. PHP programozási nyelv

A backend megvalósításához a PHP programozási nyelv került alkalmazásra. A PHP egy széles körben elterjedt, szerveroldali szkriptnyelv, amelyet eredetileg webalkalmazások fejlesztésére terveztek, és mára az interneten elérhető weboldalak és alkalmazások jelentős részének alapjául szolgál. A nyelv egyik legfontosabb jellemzője, hogy nyílt forráskódú, platformfüggetlen és könnyen integrálható különböző webserverekkel és adatbázis-kezelő rendszerekkel [12].

A PHP egy olyan környezetet biztosít, amely képes a mobilalkalmazás által küldött kérések feldolgozására, valamint az adatok strukturált kezelésére és visszaküldésére. A backend REST API formájában valósult meg, amely HTTP-alapú kommunikáción keresztül biztosít kapcsolatot az alkalmazással. Ennek köszönhetően a rendszer jól elkülöníthető komponensekre bontható, ahol a kliens az adatmegjelenítésért, míg a szerver az üzleti logikáért és adatkezelésért felel.

A PHP választását a fejlesztés során az is indokolta, hogy a backend megvalósításához a Laravel keretrendszer került felhasználásra, amely a PHP egyik legelterjedtebb és legjobban támogatott fejlesztési keretrendszere.

Összességében a PHP megfelelő alapot biztosított egy olyan backend rendszer kialakításához, amely hatékonyan képes kiszolgálni a mobilalkalmazást.

3.4. Laravel keretrendszer

A backend megvalósítása a Laravel 12 keretrendszer segítségével történt. A Laravel egy nyílt forráskódú, PHP alapú keretrendszer, amelynek célja a fejlesztési folyamat egyszerűsítése és a jól strukturált, karbantartható kód elősegítése. A keretrendszer az MVC (Model–View–Controller) architektúrális mintát követi, amely lehetővé teszi az alkalmazás különböző rétegeinek elkülönítését, így az adatkezelés, az üzleti logika és a kommunikációs réteg egymástól függetlenül kezelhető [13].

A Laravel alkalmazása több szempontból is indokolt volt a fejlesztés során. A keretrendszer számos beépített megoldást kínál, amelyek csökkentik a fejlesztési időt és növelik a kód megbízhatóságát. Ide tartozik az ORM-alapú adatkezelés, a migrációs rendszer, valamint a validációs és hibakezelési lehetőségek.

Az adatbázis-kezelés az Eloquent ORM segítségével történt, amely lehetővé tette az adatok objektumorientált kezelését, ezáltal egyszerűbbé és átláthatóbbá téve a fejlesztést. A migrációk biztosították az adatbázis szerkezetének következetes és reprodukálható kezelését, míg a beépített validáció hozzájárult a bemeneti adatok biztonságos feldolgozásához.

Ezek az eszközök együttesen lehetővé tették, hogy a fejlesztés során a hangsúly elsősorban az alkalmazás üzleti logikájának megvalósításán maradjon.

3.5. Hitelesítés és biztonság

Az alkalmazás hitelesítési rétege Bearer token alapú megoldásra épül, amely a REST API-k esetében széles körben alkalmazott megközelítés. Ennek lényege, hogy a kliens minden kérés során egy egyedi azonosító tokent küld az Authorization fejlécben, amely alapján a szerver azonosítja a felhasználót. Ez a modell a stateless kommunikációt támogatja, vagyis a szerver nem tárol hagyományos munkamenet-információt, hanem minden kérést önállóan értelmez [14].

Ezen módszer alkalmazását elsősorban az indokolta, hogy jól illeszkedik a mobilalkalmazások működéséhez és a REST alapú architektúrához. A token alapú hitelesítés lehetővé teszi, hogy a kliens és a szerver közötti kommunikáció egyszerű és hatékony maradjon, miközben a hozzáférés és ellenőrzés egységes módon valósítható meg. A Laravel keretrendszer middleware mechanizmusa biztosítja, hogy a hitelesítést igénylő végpontok csak megfelelő jogosultság esetén legyenek elérhetők, így az API védelme jól kezelhető [15].

A biztonság szempontjából fontos szerepet kap a jelszavak megfelelő kezelése. A felhasználói jelszavak nem kerülnek nyers formában tárolásra, hanem hash-elt módon, amelyhez a Laravel beépített megoldást biztosít [16]. Emellett az érzékeny adatok, például a jelszó és az azonosító token, nem jelennek meg az API válaszaiban, ami csökkenti az adatszivargás kockázatát.

A token-alapú azonosítás további előnye, hogy a felhasználói munkamenetek kezelése egyszerűen megoldható. A bejelentkezés során generált új token biztosítja, hogy a korábban kiadott azonosítók érvényüket veszítsék, ami hatékony védelmi mechanizmust jelent a jogosulatlan hozzáférésekkel szemben.

3.6. Adatbázis – MySQL

Az alkalmazás adatainak tárolása a MySQL relációs adatbázis kezelő rendszerrel történik. A MySQL egy széles körben használt, nyílt forráskódú megoldás, amely hosszú ideje stabil alapot biztosít különböző alkalmazások számára. A PHP-val és a Laravel keretrendszerrel való jó integrációja miatt gyakran alkalmazzák backend rendszerek esetében. Az adatkezelés megbízhatóságát az ACID (Atomicity, Consistency, Isolation, Durability) elvű működés biztosítja, amely garantálja, hogy az adatbázis műveletek konzisztensen és biztonságosan hajtsódjanak végre, ami különösen fontos a felhasználói adatok és a rendszerben kezelt értékek esetén [17].

A választás mellett szólt az is, hogy a MySQL jól illeszkedik az alkalmazás jelenlegi méretéhez. Egy Laravel környezetben végzett összehasonlító vizsgálat szerint kisebb és közepes adatmennyiség mellett is stabil és megfelelő teljesítményt nyújt, így a rendszer jelenlegi terhelési igényeihez megfelelő megoldást nyújt [18].

Az adatbázis felépítése nem manuálisan történt, hanem Laravel migrációk segítségével. Ez azt jelenti, hogy az adatbázis szerkezete kódban van leírva, és minden változtatás külön fájlban kerül rögzítésre. Ennek előnye, hogy a séma könnyen követhető, és bármikor újra létrehozható egyetlen paranccsal, ami a fejlesztés során kifejezetten hasznos [19].

Az alkalmazás adatmodellje több, egymással kapcsolatban álló entitásból épül fel, például felhasználók, aktivitási adatok, étkezési naplók és XP-hez kapcsolódó bejegyzések. Ezek külön táblákban kerülnek tárolásra, és relációkon keresztül kapcsolódnak egymáshoz. Az adatok közötti kapcsolatok kezelését az Eloquent ORM segíti, amely lehetővé teszi, hogy az adatbázis-kezelés objektumorientált módon történjen, így a lekérdezések átláthatóbbak és könnyebben kezelhetők. Az ORM használatával az adatbázis táblák és azok kapcsolatai modellek formájában jelennek meg, ami csökkenti a közvetlen SQL lekérdezések

szükségességét, és egységesebb adatkezelést tesz lehetővé az alkalmazás különböző részein [20].

Összességében a MySQL és a Laravel migrációs rendszere együtt egy jól kezelhető és megbízható adatkezelési alapot biztosít az alkalmazás számára.

3.7. Frontend fejlesztői környezet – Android Studio

A mobilalkalmazás fejlesztése az Android Studio integrált fejlesztői környezetben történt. Az Android Studio a Google által fejlesztett és karbantartott hivatalos IDE Android alkalmazások készítéséhez, amely az IntelliJ IDEA platformra épül, és egységes környezetet biztosít a fejlesztési folyamat minden lépéséhez [21]. Az eszköz tartalmazza a kódszerkesztőt, a Kotlin-fordítót, a Gradle build rendszert, valamint a hibakereséshez és teszteléshez szükséges eszközöket is.

A fejlesztés során különösen hasznosnak bizonyult a beépített Android Virtual Device (AVD) emulátor, amely lehetővé teszi az alkalmazás futtatását és tesztelését fizikai eszköz nélkül, különböző Android verziókat és eszköztípusokat szimulálva. Ez jelentősen megkönnyítette a fejlesztési folyamatot, mivel az alkalmazás működése több környezetben is gyorsan ellenőrizhető volt. Az emulátor segítségével a kliens és a backend közötti kommunikáció is tesztelhető volt fejlesztői környezetben.

Az Android Studio a Gradle build rendszert használja, amely a projekt felépítését és a szükséges függőségek kezelését biztosítja. A fejlesztés során a konfiguráció Kotlin DSL formában történt, ami átláthatóbbá tette a build fájlokat és jobban illeszkedett a Kotlin alapú környezethez. A Jetpack Compose használata szintén megkönnyítette a felület kialakítását, mivel a komponensek már szerkesztés közben is megjeleníthetők voltak, így gyorsabban lehetett ellenőrizni a változtatások eredményét.

Összességében az Android Studio egy jól használható és egységes fejlesztői környezetet biztosított, amely a mobilalkalmazás fejlesztésének minden fontos lépését lefedte. Az integrált eszközöknek köszönhetően a fejlesztés, tesztelés és hibakeresés egy helyen valósulhatott meg, ami átláthatóbbá és hatékonyabbá tette a teljes folyamatot.

3.8. Kotlin programozási nyelv

A mobilalkalmazás Kotlin programozási nyelven készült. A Kotlin egy modern, statikusan típusos nyelv, amelyet a JetBrains fejlesztett, és amelyet a Google 2019-ben az Android fejlesztés elsődleges nyelvévé tett [22]. A statikusan típusos működés azt jelenti, hogy a változók típusát már fordítási időben ellenőrzi a rendszer, így sok hiba még a program futtatása előtt kiszűrhető.

A Kotlin használata a fejlesztés során elsősorban a kód egyszerűsítésében jelentett előnyt. Például az alkalmazásban az API kommunikációhoz szükséges adatmodellek data class-okkal kerültek megvalósításra, amelyek egyetlen sorban tartalmazzák az adatmezőket, miközben a nyelv automatikusan létrehozza a szükséges segédmetódusokat. Ez jelentősen csökkenti a manuálisan megírandó kód mennyiségét a Java megoldásokhoz képest, ahol ezek külön implementációt igényelnének.

A null-biztonság szintén fontos szerepet játszott. A Kotlin megköveteli, hogy külön jelöljük azokat a változókat, amelyek felvehetnek null értéket, így a hiányzó adatok kezelése nem maradhat ki a programból. Ez különösen a backendtől érkező válaszok feldolgozásánál hasznos, ahol nem minden mező garantáltan kitöltött. Ennek köszönhetően csökkenthető a futásidejű hibák száma.

A Kotlin jól illeszkedik az Android fejlesztési környezethez is. A Jetpack Compose használata során a felhasználói felület közvetlenül Kotlin kódban kerül megvalósításra, ami egységesebbé teszi a fejlesztést, és gyorsabb módosításokat tesz lehetővé. A felületi elemek és az alkalmazás logikája így közelebb kerül egymáshoz, ami átláthatóbb szerkezetet eredményez.

Összességében a Kotlin használata hozzájárult ahhoz, hogy a kód tömörebb, átláthatóbb és biztonságosabb legyen. A nyelv tulajdonságai lehetővé tették, hogy a fejlesztés során inkább az alkalmazás működésére lehessen koncentrálni, mint a technikai részletek kezelésére.

3.9. Jetpack Compose

A mobilalkalmazás felhasználói felülete Jetpack Compose segítségével készült. Ez egy modern, Kotlin alapú UI keretrendszer, amely az utóbbi években vált az Android fejlesztés egyik meghatározó eszközévé. A Compose működésének alapja a deklaratív szemlélet, ami

azt jelenti, hogy a felület mindig az aktuális állapot alapján épül fel. Ha az adatok változnak, a rendszer automatikusan frissíti az érintett elemeket, így a fejlesztőnek nem kell külön foglalkoznia a manuális frissítésekkel.

Ez a megközelítés jól passzolt az alkalmazás felépítéséhez, mivel több olyan képernyő is van, ahol a backendtől érkező adatok határozzák meg a megjelenést. A Compose használatával ezek a változások közvetlenül megjelennek a felületen, ami egyszerűbbé tette a logika és a UI összekapcsolását, és csökkentette a szükséges kód mennyiségét a korábbi XML alapú megoldásokhoz képest [23].

A képernyők külön függvényekként vannak felépítve, ami átláthatóbb struktúrát eredményez. A felület elemei kisebb, újrafelhasználható komponensekre lettek bontva, például kártyákra, listákra vagy párbeszédablakokra. Ez nemcsak a kód tisztaságát javítja, hanem megkönnyíti a későbbi módosításokat is.

Az állapotkezelés a Compose egyik kulcseleme, ami a fejlesztés során is fontos szerepet kapott. Az olyan megoldások, mint a 'remember' vagy a 'mutableStateOf' biztosítják, hogy a felület mindig az aktuális adatoknak megfelelően jelenjen meg. Ez különösen hasznos volt a betöltési állapotok, hibaüzenetek és felhasználói interakciók kezelésénél. Emellett a 'MaterialTheme' használata egységes megjelenést biztosít az alkalmazás teljes felületén.

Összességében a Jetpack Compose használata jól illeszkedett az alkalmazás jellegéhez. Egyszerűbbé tette a felület kezelését, átláthatóbbá a kódot, és lehetővé tette, hogy a fejlesztés során inkább a működésre lehessen koncentrálni.

3.10. Hálózati kommunikáció

A mobilalkalmazás és a backend közötti kommunikáció Retrofit2 és OkHttp könyvtárak segítségével valósul meg. A két technológia egymásra épül, és együtt biztosítanak egy jól kezelhető megoldást a REST API-k elérésére.

Az OkHttp egy alacsony szintű HTTP kliens, amely a tényleges hálózati kommunikációért felel. Feladata a kérések elküldése, a válaszok fogadása, valamint a kapcsolatkezelés optimalizálása. Az egyik fontos jellemzője az interceptor mechanizmus, amely lehetővé teszi, hogy a kimenő kérések módosíthatók legyenek, például fejlécek automatikus hozzáadásával

vagy naplózással [24]. Ez a fejlesztés során különösen hasznos volt a hitelesítés kezelésében és a hibakeresésben.

A Retrofit erre a működésre épül, és egy magasabb szintű absztrakciót biztosít. Segítségével az API végpontok egy Kotlin interfészben, annotációk segítségével definiálhatók. Ez azt jelenti, hogy nem szükséges manuálisan HTTP kéréseket összeállítani, hanem elegendő a végpontok leírása, a könyvtár pedig automatikusan kezeli a kommunikációt. A válaszok feldolgozása szintén egyszerűbbé válik, mivel a JSON formátumú adatok közvetlenül Kotlin adatosztályokra képezhetők le [25].

Az alkalmazásban a hálózati réteg több elkülönített komponensből épül fel. Az API végpontokat egy interfész tartalmazza, a kliens konfigurációja egy központi osztályban történik, míg a hitelesítés interceptor segítségével valósul meg. Ez az interceptor minden kéréshez automatikusan hozzáadja az azonosításhoz szükséges tokent, így ezzel a felülethez tartozó kódban nem kell külön foglalkozni. A token helyi tárolása biztosítja, hogy az alkalmazás újraindítása után is elérhető maradjon.

A hálózati hívások aszinkron módon történnek, így nem blokkolják a felhasználói felület működését. Ez lehetővé teszi, hogy az alkalmazás folyamatosan reagálni tudjon a felhasználói műveletekre, miközben a háttérben zajlik az adatlekérés vagy küldés.

Összességében a Retrofit és az OkHttp kombinációja egy átlátható és megbízható megoldást nyújt a kliens–szerver kommunikáció megvalósítására, amely jól illeszkedik az alkalmazás architektúrájához.

3.11. A rendszer architektúrájának összefoglalása

Az eddig bemutatott technológiák egy egységes, kliens–szerver architektúrában állnak össze, amely meghatározza az alkalmazás teljes működését. A rendszer két fő részből épül fel: egy Laravel alapú backendből és egy Android mobilalkalmazásból, amelyek egymással REST API-n keresztül kommunikálnak JSON formátumban.

A szerver oldalon a kérések feldolgozása jól elkülönített rétegek mentén történik. A bejövő HTTP kérés az útvonalkezelésen keresztül a megfelelő vezérlőhöz kerül, ahol az üzleti logika fut le, majd az adatbázis-műveletek az Eloquent modelleken keresztül valósulnak meg. A

válasz egységesen JSON formátumban kerül visszaküldésre, ami egyszerűvé és kiszámíthatóvá teszi a kliens oldali feldolgozást.

A mobilalkalmazás oldalán a felhasználói műveletek indítják el a hálózati hívásokat, amelyek Retrofit és OkHttp segítségével jutnak el a backendhez. A visszakapott adatok a Compose állapotkezelésén keresztül jelennek meg a felületen, így a UI automatikusan frissül, amikor az állapot megváltozik.

A választott architektúra egyik legnagyobb előnye a két oldal közötti laza csatolás. A frontend és a backend egymástól függetlenül fejleszthető, ami megkönnyíti a karbantartást és a későbbi bővítést. Emellett a REST API használata lehetővé teszi, hogy a rendszer más kliensekkel is könnyen összekapcsolható legyen.

Összességében a kialakított rendszer egy jól strukturált, átlátható és bővíthető megoldást ad, amely stabil alapot biztosít a további fejlesztésekhez.

4. A LvlUp! Mobilalkalmazás bemutatása

4.1. Az alkalmazás felépítése és navigációja

Az alkalmazás több, egymással összekapcsolt képernyőből áll, amelyek különböző funkciókat valósítanak meg. A felhasználó ezek között a képernyők között tud navigálni attól függően, hogy éppen milyen műveletet szeretne végrehajtani, például aktivitást rögzíteni, étkezést naplózni vagy akár statisztikákat megtekinteni.

A működés szempontjából a fő belépési pont a bejelentkezés, amely után a felhasználó a főképernyőre kerül. Innen érhetők el a további funkciók, amelyek külön képernyőkön jelennek meg. Az alkalmazás fő részei a következők:

- bejelentkezési és regisztrációs felület,
- főképernyő,
- aktivitás kezelése,
- táplálkozás kezelése,
- heti statisztika megjelenítése,
- ranglista rendszer

Az alkalmazás belépési pontja a „MainActivity.kt” fájl amely az Android alkalmazás indulásakor inicializálja a teljes felhasználói felületet. Ebben az osztályban történik a különböző képernyők közötti navigáció kezelése is.

A felület felépítése a „setContent” blokkban történik, ahol az alkalmazás különböző állapotváltozók segítségével dönti el, hogy éppen melyik képernyőt kell megjeleníteni (1. ábra).

```

3 Usages
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        setContent {
            var showLogin by remember { mutableStateOf( value = true) }
            var showRegister by remember { mutableStateOf( value = false) }
            var showProfile by remember { mutableStateOf( value = false) }
            var showActivity by remember { mutableStateOf( value = false) }
            var showConsumption by remember { mutableStateOf( value = false) }
            var showWeeklyStats by remember { mutableStateOf( value = false) }
            var showLeaderboards by remember { mutableStateOf( value = false) }
            var showLeaderboardDetail by remember { mutableStateOf( value = false) }
            var selectedLeaderboardId by remember { mutableIntStateOf( value = 0) }
        }
    }
}

```

1. Ábra: Felület felépítése: lehetséges képernyőállapotok

A navigáció konkrét megvalósítása egy „when” szerkezet segítségével történik, ahol az alkalmazás az aktuális állapot alapján választja ki a megjelenítendő képernyőt (2. és 3. ábra).

```

Gamifikalt_Fitnessz_AlkalmazasTheme {
    when {
        showLogin -> {
            LoginScreen(
                onLoginSuccess = {
                    showLogin = false
                    showRegister = false
                    showProfile = false
                    showActivity = false
                    showConsumption = false
                    showWeeklyStats = false
                    showLeaderboards = false
                    showLeaderboardDetail = false
                },
                onRegisterClick = {
                    showLogin = false
                    showRegister = true
                }
            )
        }
    }
}

```

2. Ábra: Képernyőállapotok beállítása

```

else -> {
    HomeScreen(
        onProfileClick = { showProfile = true },
        onActivityClick = { showActivity = true },
        onConsumptionClick = { showConsumption = true },
        onWeeklyStatsClick = { showWeeklyStats = true },
        onLeaderboardsClick = { showLeaderboards = true },
        onLogout = {
            TokenStore.clear( context = this@MainActivity)
            showProfile = false
            showRegister = false
            showActivity = false
            showConsumption = false
            showWeeklyStats = false
            showLeaderboards = false
            showLeaderboardDetail = false
            showLogin = true
        }
    )
}

```

3. Ábra: Képernyőállapotok beállítása

A „when” szerkezet további ágai is hasonló logika szerint működnek, vagyis minden esetben egy adott képernyő megjelenítéséhez tartozó állapotváltozó értéke határozza meg, hogy a felhasználó melyik felületet látja. Ennek köszönhetően az alkalmazás navigációja egyszerűen

követhető, mivel minden képernyő megjelenítése egy jól elkülöníthető logikai feltételhez kapcsolódik.

A navigáció ilyen megoldása a fejlesztés során kifejezetten előnyösnek bizonyult, mivel az egyes funkciók jól elkülönültek egymástól. Ez nemcsak átláthatóbbá tette a kódot, hanem azt is megkönnyítette, hogy később új elemeket adjak hozzá.

Összességében az alkalmazás felépítése jól tagolt, mivel minden képernyő külön komponensként működik, a közöttük történő váltást pedig a „MainAcitivity” vezérli. A következő alfejezetekben ezeknek az egyes funkcióknak a részletes bemutatása következik.

4.2. Felhasználókezelés és hitelesítés

Az alkalmazás használatához a felhasználónak először regisztrálnia kell (4. ábra), majd ezt követően be kell jelentkeznie (5. ábra). A felhasználókezelés célja nem csupán az azonosítás, hanem a személyre szabott működés megalapozása is, mivel több funkció, például a kalóriacélok meghatározása vagy a napi kihívások egy része a megadott alapadatokra épül.

Regisztráció

E-mail

Felhasználónév

Jelszó

Min. 8 karakter

Jelszó újra

Nem

Férfi Nő

Kiválasztott nem

Kor

Magasság (cm)

Súly (kg)

Regisztráció

4. Ábra: Regisztrációs képernyő

LvlUp!

Email

Jelszó

Bejelentkezés

Még nincs profilod? Regisztrálj

5. Ábra: Bejelentkezési képernyő

A frontend oldalon a regisztráció egy űrlapon keresztül valósul meg, ahol a felhasználó a szükséges mezőket kitölti (6. ábra), majd az alkalmazás ezeket egy objektumba rendezi, és továbbítja a backend felé.

```

@Composable
fun RegistrationScreen(
    onRegisterSuccess: () -> Unit = {},
    onBackClick: () -> Unit = {}
) {
    val context = LocalContext.current
    val scrollState = rememberScrollState()
    val scope = rememberCoroutineScope()

    var email by remember { mutableStateOf( value = "" ) }
    var username by remember { mutableStateOf( value = "" ) }
    var password by remember { mutableStateOf( value = "" ) }
    var passwordAgain by remember { mutableStateOf( value = "" ) }
    var gender by remember { mutableStateOf( value = "" ) }
    var age by remember { mutableStateOf( value = "" ) }
    var height by remember { mutableStateOf( value = "" ) }
    var weight by remember { mutableStateOf( value = "" ) }

```

6. Ábra: Regisztrációs űrlap változói frontend oldalon

A backend oldalon a felhasználói adatok szerkezetét a felhasználó modell írja le, amely tartalmazza a működés szempontjából fontos mezőket (7. ábra).

```

class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
        'api_token',
        'gender',
        'age',
        'height',
        'weight',
        'goal_type',
        'calorie_target',
        'xp',
        'current_streak',
        'best_streak',
    ];
}

```

7. Ábra: A User model szerkezete

A regisztráció beküldése után a backend ellenőrzi a megadott adatokat (9. ábra), majd létrehozza a felhasználót (8. ábra). A jelszó biztonsági okból nem sima szöveggént kerül eltárolásra, hanem titkosított formában.

```
public function register(Request $request)
{
    $validator = Validator::make($request->all(), [
        'name' => 'required|string|max:255',
        'email' => 'required|string|email|max:255|unique:users,email',
        'password' => 'required|string|min:8',
        'gender' => 'required|in:male,female',
        'age' => 'required|integer|min:1|max:120',
        'height' => 'required|integer|min:50|max:250',
        'weight' => 'required|numeric|min:20|max:500',
    ]);
}
```

9. Ábra: Validátor a felhasználó adataihoz

```
$user = User::create([
    'name' => $request->name,
    'email' => $request->email,
    'password' => Hash::make($request->password),
    'api_token' => Str::random(length: 60),
    'gender' => $request->gender,
    'age' => $request->age,
    'height' => $request->height,
    'weight' => $request->weight,
]);
```

8. Ábra: Felhasználó létrehozása

A felhasználó a sikeres regisztrációt követően bejelentkezhet az alkalmazásba az e-mail címe és jelszava megadásával. A bejelentkezés során a frontend egy API hívást indít, amely elküldi ezeket az adatokat a backend számára (10. ábra).

```

val api = ApiClient.create(context)

api.login( req = LoginRequest(email.trim(), password))
    .enqueue( callback = object : Callback<AuthResponse> {

        override fun onResponse(
            call: Call<AuthResponse>,
            response: Response<AuthResponse>
        ) {

            loading = false

            if (response.isSuccessful && response.body() != null) {
                TokenStore.save(context, response.body()!!.token)
                onLoginSuccess()
            } else {
                errorText = "Hibás email vagy jelszó"
            }
        }
    })

```

10. Ábra: API hívás a bejelentkezéshez

A backend a bejelentkezési kérés feldolgozása során ellenőrzi, hogy a megadott e-mail címhez tartozik-e felhasználó, valamint hogy a jelszó helyes-e. Sikeres ellenőrzés esetén egy token kerül visszaadásra, amely a későbbi kérések hitelesítésére szolgál (11. ábra).

```

$user = User::where('email', $request->email)->first();

if (!$user || !Hash::check($request->password, $user->password)) {
    return response()->json([
        'message' => 'Invalid credentials',
    ], status: 401);
}

$user->api_token = Str::random( length: 60);
$user->save();

return response()->json([
    'user' => $user,
    'token' => $user->api_token,
]);

```

11. Ábra: Email és jelszó ellenőrzés backend oldalon

A kapott token kliens oldalon kerül eltárolásra, így az alkalmazás a későbbi műveletek során is hozzá tud férni (12. ábra).

```

fun save(context: Context, token: String) {
    val prefs = context.getSharedPreferences( name = PREFS, mode = Context.MODE_PRIVATE)
    prefs.edit().putString(KEY, token).apply()
}

fun get(context: Context): String? {
    val prefs = context.getSharedPreferences( name = PREFS, mode = Context.MODE_PRIVATE)
    return prefs.getString(KEY, defValue = null)
}

2 Usages
fun clear(context: Context) {
    val prefs = context.getSharedPreferences( name = PREFS, mode = Context.MODE_PRIVATE)
    prefs.edit().remove(KEY).apply()
}

```

12. Ábra: Műveletek a token-nel

A további API hívások során a token automatikusan hozzáadásra kerül a kérésekhez, így a backend minden esetben azonosítani tudja a felhasználót (13. ábra).

```

class AuthInterceptor(private val context: Context) : Interceptor {
    override fun intercept(chain: Interceptor.Chain): Response {
        val original = chain.request()
        val token = TokenStore.get(context)

        if (token.isNullOrEmpty()) {
            return chain.proceed( request = original)
        }

        val newReq = original.newBuilder()
            .header( name = "Authorization", value = "Bearer $token")
            .build()

        return chain.proceed( request = newReq)
    }
}

```

13. Ábra: Token hozzárendelés API hívásnál

A backend oldalon a beérkező kérések feldolgozása során egy middleware végzi a token ellenőrzését, és ennek alapján határozza meg az aktuális felhasználót (14. ábra).

```

class ApiTokenAuth
{
    public function handle(Request $request, Closure $next)
    {
        $header = $request->header('Authorization');

        if (!$header || !str_starts_with($header, 'Bearer ')) {
            return response()->json(['message' => 'Unauthorized'], status: 401);
        }

        $token = substr($header, offset: 7);

        $user = User::where('api_token', $token)->first();

        if (!$user) {
            return response()->json(['message' => 'Unauthorized'], status: 401);
        }

        $request->merge(['auth_user' => $user]);

        return $next($request);
    }
}

```

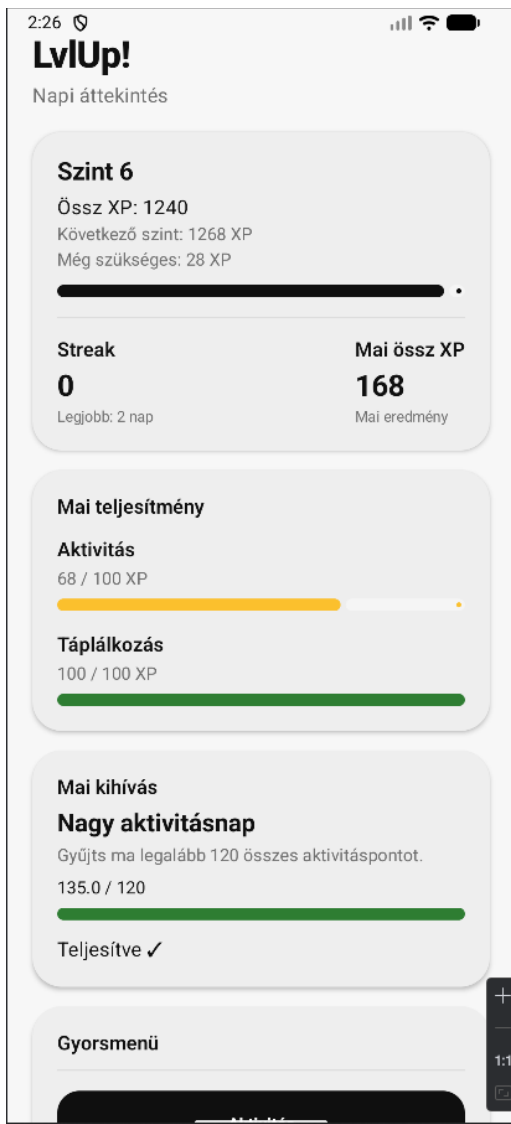
14. Ábra: Token ellenőrzés API hívásnál

A felhasználókezelés tehát egy token alapú hitelesítési mechanizmusra épül, ahol a frontend az adatbekérésért és a kommunikációért felel, míg a backend végzi az ellenőrzést és az azonosítást. Ennek köszönhetően az alkalmazás funkciói biztonságosan és felhasználónként elkülönítve használhatók.

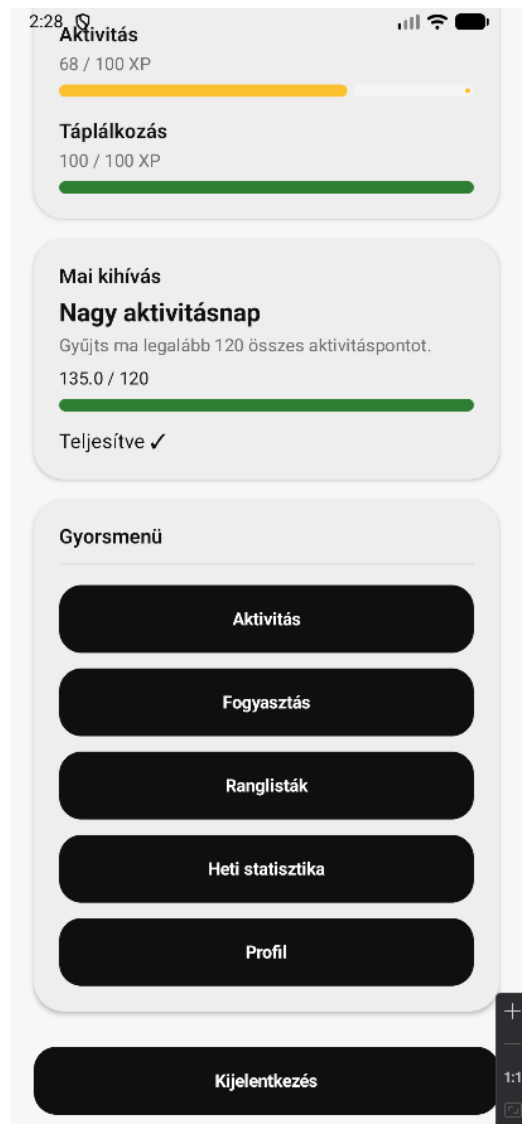
4.3. Főképernyő

Az alkalmazás központi eleme a főképernyő, amely egy áttekintő felületet biztosít a napi állapotról, valamint gyors hozzáférést nyújt a legfontosabb funkciókhoz. A bejelentkezést követően a felhasználó közvetlenül erre a képernyőre kerül, így ez tekinthető az alkalmazás kiindulópontjának (15. és 16. ábra).

A főképernyő kártyákra bontva jeleníti meg az információkat, így az egyes funkcionális elemek jól elkülönülnek egymástól, és könnyen áttekinthetők (15. és 16. ábra).



15. Ábra: Az alkalmazás főképernyője



16. Ábra: Az alkalmazás főképernyője

A képernyő felső részén található a felhasználó aktuális szintjét megjelenítő kártya. Ez a blokk tartalmazza az összesített XP értéket, valamint azt is, hogy mennyi szükséges a következő szint eléréséhez. Emellett itt jelenik meg a felhasználó aktuális sorozata (streak), valamint a napi összesített XP érték (17. ábra).

Frontend oldalon az általam definiált 'DashboardCard' komponens használata egy jól átlátható, letisztult megoldást nyújtott az információk megjelenítéséhez (18. és 19. ábra).

```

DashboardCard {
    Text(
        text = "Szint ${xp.level}",
        style = MaterialTheme.typography.titleLarge
    )

    Spacer(modifier = Modifier.height( height = 8.dp))

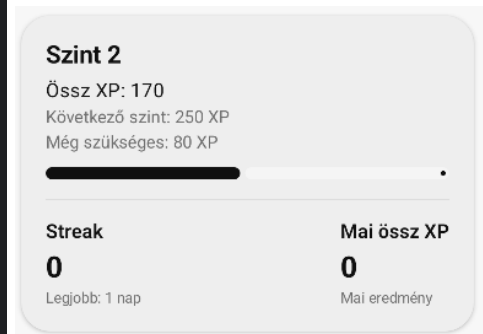
    Text(
        text = "Össz XP: ${xp.xp}",
        style = MaterialTheme.typography.bodyLarge
    )

    Spacer(modifier = Modifier.height( height = 4.dp))

    Text(
        text = "Következő szint: ${xp.next_level_xp} XP",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )

    Spacer(modifier = Modifier.height( height = 4.dp))
}

```



17. Ábra: Fejlődést mutató kártya a főképernyőn

18. Ábra: DashboardCard megvalósítás a szintkövetéshez

A következő kártya a napi teljesítményt mutatja be két külön bontásban: aktivitás és táplálkozás. Mindkét esetben egy progress bar szemlélteti, hogy a felhasználó mennyire teljesítette az adott napi célt (20. ábra).

A vizuális megjelenítés segíti a gyors értelmezést, mivel a felhasználó azonnal láthatja, hogy mely területen van még elmaradása.

```

DashboardCard {
    Text(
        text = "Mai teljesítmény",
        style = MaterialTheme.typography.titleMedium
    )

    Spacer(modifier = Modifier.height( height = 16.dp))

    Text(
        text = "Aktivitás",
        style = MaterialTheme.typography.titleMedium
    )

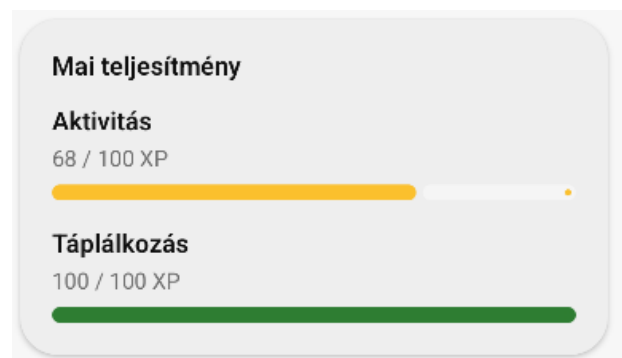
    Spacer(modifier = Modifier.height( height = 6.dp))

    Text(
        text = "${xp.today_activity_xp} / 100 XP",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )

    Spacer(modifier = Modifier.height( height = 8.dp))

    LinearProgressIndicator(
        progress = { activityProgress },
        modifier = Modifier
            .fillMaxWidth()
            .height( height = 10.dp),
        color = progressColor(activityProgress),
        trackColor = MaterialTheme.colorScheme.surfaceVariant
    )
}

```

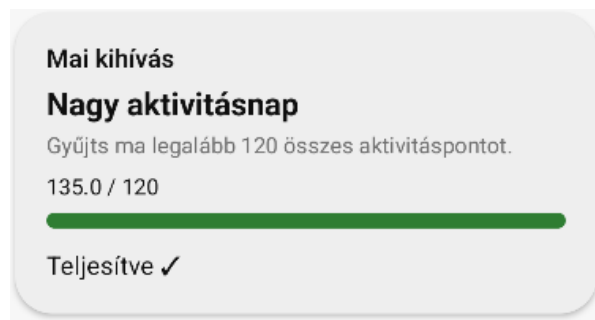


20. Ábra: Teljesítménykövető kártya a főképernyőn

19. Ábra: DashboardCard megvalósítás a teljesítménykövetéshez

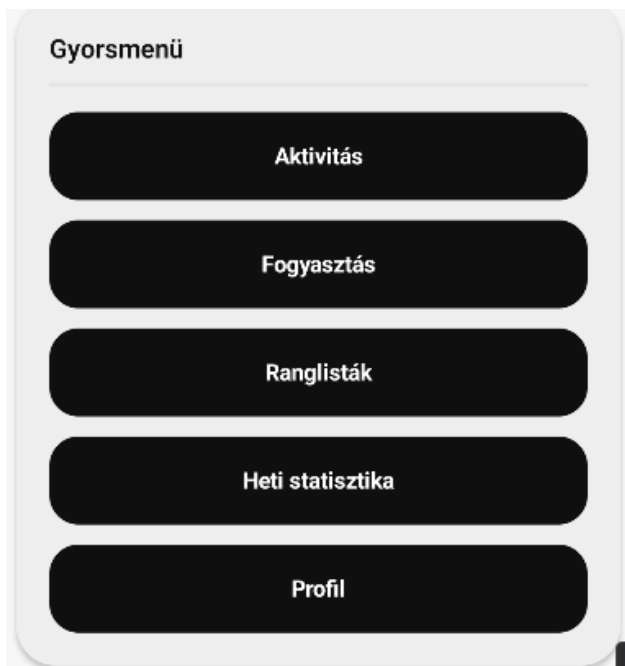
A főképernyő egy külön kártyán jeleníti meg az aktuális napi kihívást. Itt látható a kihívás neve, rövid leírása, valamint a teljesítés állapota egy progress bar segítségével. Ez a blokk folyamatos visszajelzést ad arról, hogy a felhasználó mennyire halad az adott napi feladattal, így motiválja a cél teljesítésére (21. ábra).

A megjelenítés kialakítása megegyezik a korábban bemutatott kártyákkal, vagyis ez az elem is a 'DashboardCard' komponens használatával valósul meg, és hasonló módon épül fel, mint a napi teljesítményt megjelenítő blokk



21. Ábra: Kihívás kártya a főképernyőn

A képernyő alsó részén található gyorsmenü biztosítja a navigációt az alkalmazás többi funkciója felé (23. ábra). Innen közvetlenül elérhetők az olyan képernyők, mint az aktivitás rögzítése, a fogyasztás naplózása, a ranglisták, a heti statisztika, illetve a felhasználó itt tudja módosítani a profiljához tartozó adatokat is (22. ábra), például az életkorát, magasságát és testsúlyát.



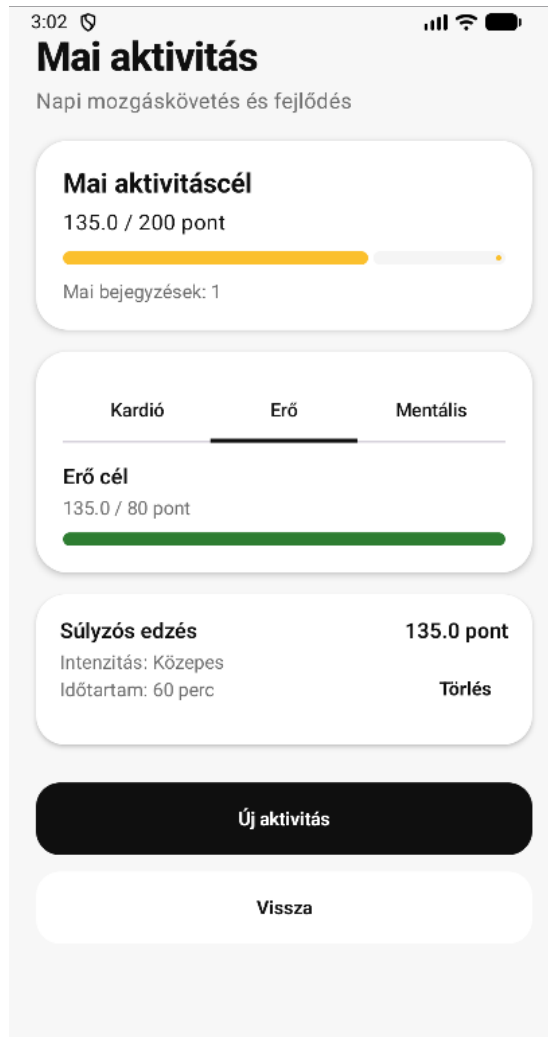
23. Ábra: Gyorsmenü a főképernyőn

22. Ábra: Profil képernyő

Az egységes megjelenést az biztosítja, hogy a főképernyő minden eleme a 'DashboardCard' komponens segítségével kerül kialakításra, így a különböző információs blokkok azonos struktúrában és stílusban jelennek meg. Ez hozzájárul az átlátható és könnyen használható felület kialakításához.

4.4. Aktivitás kezelése

Az alkalmazás egyik alapvető funkciója az aktivitások rögzítése, amely lehetővé teszi a felhasználó számára, hogy nyomon kövesse a napi mozgását. Az aktivitások rögzítése hozzájárul a napi teljesítmény kiszámításához, valamint a felhasználó előrehaladásának követéséhez.



24. Ábra: Aktivitás képernyő

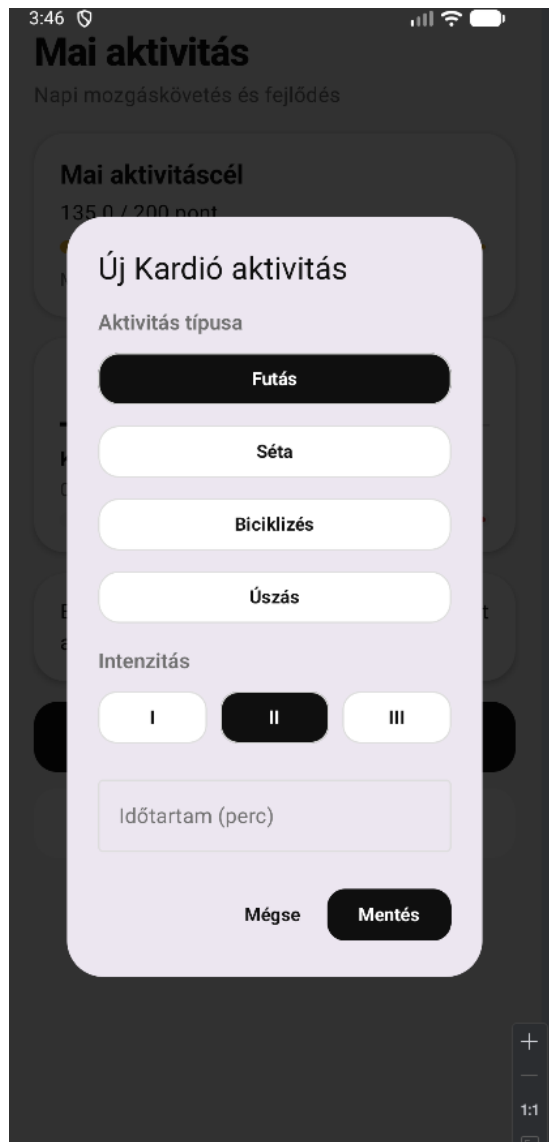
A képernyő felső részén a napi aktivitási cél jelenik meg, amely egy 'progress bar' segítségével mutatja, hogy a felhasználó mennyit teljesített az adott napi célból. Emellett itt látható a napi bejegyzések száma is. A képernyő középső részén kategóriák szerint bontva jelennek meg az aktivitások, míg az alsó részen az adott naphoz tartozó egyéni bejegyzések láthatók. Az egyes elemeknél feltüntetésre kerül az aktivitás típusa, az intenzitás, az időtartam, valamint a hozzájuk tartozó pontérték. A képernyő alján található gomb segítségével új aktivitás rögzíthető (24. ábra).

Az aktivitás képernyő több, egymástól jól elkülönülő kártyából épül fel (25. ábra). A napi cél, a kategória alapú bontás és a rögzített aktivitások listája külön blokkokban jelennek meg, ami átláthatóbbá teszi a felületet.

```
ActivityDashboardCard {  
    Text(  
        text = "Mai aktivitáscél",  
        style = MaterialTheme.typography.titleLarge  
    )  
  
    Spacer(modifier = Modifier.height( height = 8.dp))  
  
    Text(  
        text = "$totalPoints / $totalGoal pont",  
        style = MaterialTheme.typography.bodyLarge  
    )  
  
    Spacer(modifier = Modifier.height( height = 12.dp))  
  
    LinearProgressIndicator(  
        progress = { totalProgress },  
        modifier = Modifier  
            .fillMaxWidth()  
            .height( height = 10.dp),  
        color = activityProgressColor(totalProgress),  
        trackColor = MaterialTheme.colorScheme.surfaceVariant  
    )  
  
    Spacer(modifier = Modifier.height( height = 12.dp))  
  
    Text(  
        text = "Mai bejegyzések: ${summary?.activity_count ?: 0}",  
        style = MaterialTheme.typography.bodyMedium,  
        color = MaterialTheme.colorScheme.onSurfaceVariant  
    )  
}
```

25. Ábra: Egy kártya megvalósítása az aktivitás képernyőn

Új aktivitás rögzítése egy felugró ablakban történik, amely az „Új aktivitás” gomb megnyomásával érhető el. A felhasználó itt először kiválasztja az aktivitás típusát, majd megadja az intenzitást és az időtartamot (26. ábra).



26. Ábra: Új aktivitás rögzítéséhez felugró ablak

A felületen az aktivitás típusa előre definiált opciók közül választható ki, így elkerülhető a szabad szöveges bevitelből adódó hibalehetőség. Az intenzitás szintén előre meghatározott értékekkel adható meg, az időtartam pedig numerikus mezőben kerül megadásra (26. ábra).

Az aktivitás rögzítéséhez szükséges adatok a frontend oldalon állapotváltozóknak kerülnek tárolásra (27. ábra). Ezek az értékek a felhasználói interakciók során folyamatosan frissülnek, majd a mentés pillanatában kerülnek összeállításra.

```

var selectedType by remember( key1 = category) { mutableStateOf( value = activityOptions.firstOrNull() ?: "") }
var selectedIntensity by remember { mutableStateOf( value = "medium") }
var durationText by remember { mutableStateOf( value = "") }

```

27. Ábra: Állapotváltozók az aktivitáshoz

A mentés előtt az alkalmazás ellenőrzi, hogy minden szükséges adat meg lett-e adva, és hogy az időtartam érvényes szám-e (28. ábra).

```

confirmButton = {
    Button(
        onClick = {
            typeError = null
            durationError = null

            val duration = durationText.toIntOrNull()
            var valid = true

            if (selectedType.isBlank()) {
                typeError = "Válassz aktivitástípust"
                valid = false
            }

            if (duration == null || duration <= 0) {
                durationError = "Adj meg érvényes időtartamot"
                valid = false
            }

            if (!valid) return@Button

            onSave( activityType = selectedType, intensity = selectedIntensity, durationMinutes = duration!!)
        },
        shape = RoundedCornerShape( size = 14.dp)
    )
    Text( text = "Mentés")
}

```

28. Ábra: Az aktivitás érvényességének ellenőrzése

A frontend az összegyűjtött adatokat egy API hívás segítségével küldi el a backend számára. A kérés tartalmazza az aktivitás típusát, intenzitását és időtartamát (29. ábra).

```

ApiClient.create(context)
    .createActivity(
        request = CreateActivityRequest(
            category = selectedCategory,
            activity_type = activityType,
            intensity = intensity,
            duration_minutes = durationMinutes
        )
    )

```

29. Ábra: API hívás egy aktivitás rögzítéséhez

A backend oldalon az aktivitás rögzítése vezérlőn keresztül történik. A beérkező adatok először validálásra kerülnek, így a rendszer csak megfelelő formátumú és megengedett értékeket fogad el (30. ábra).

```
$validated = $request->validate([
    'category' => 'required|in:cardio,strength,mental',
    'activity_type' => 'required|string',
    'intensity' => 'required|in:low,medium,high',
    'duration_minutes' => 'required|integer|min:1|max:600',
]);
```

30. Ábra: Validátor egy aktivációhoz

A validáció után a rendszer létrehozza az új aktivitási rekordot, amelyet az aktuális felhasználóhoz kapcsol, és eltárolja a hozzá tartozó adatokat, beleértve a kiszámított pontértéket és a rögzítés dátumát is (31. ábra).

```
$activityLog = ActivityLog::create([
    'user_id' => $user->id,
    'category' => $validated['category'],
    'activity_type' => $validated['activity_type'],
    'intensity' => $validated['intensity'],
    'duration_minutes' => $validated['duration_minutes'],
    'points' => round($points, precision: 1),
    'activity_date' => Carbon::today()->toDateString(),
]);
```

31. Ábra: Aktivitás létrehozása

Ez a megoldás biztosítja, hogy minden aktivitás külön bejegyzésként kerüljön tárolásra, és egyértelműen az adott felhasználóhoz tartozzon.

Az aktivitásokhoz pontérték tartozik, amely az időtartam, az intenzitás és az aktivitás típusa alapján kerül meghatározásra. Ez a pontszámítás azért fontos, mert erre épül a napi aktivitási teljesítmény megjelenítése, valamint a későbbi gamifikációs elemek egy része is.

A számítás során a backend külön szorzókat rendel az intenzitási szintekhez és az egyes aktivitástípusokhoz, majd ezeket az időtartammal szorozza össze (32. ábra).

```

$intensityMultipliers = [
  'low' => 1.0,
  'medium' => 1.5,
  'high' => 2.0,
];

$typeMultipliers = [
  'running' => 1.4,
  'walking' => 1.0,
  'cycling' => 1.2,
  'swimming' => 1.5,
  'weightlifting' => 1.5,
  'bodyweight' => 1.2,
  'meditation' => 1.0,
  'yoga' => 1.2,
  'reading' => 0.7,
];

```

32. Ábra: Szorzók a különböző aktivitásokhoz és intenzitásokhoz

A tényleges pontérték ezek alapján az aktivitás időtartamából, az intenzitás szorzójából és az aktivitástípushoz tartozó szorzóból számolódik (33. ábra).

```

$points = $validated['duration_minutes']
  * $intensityMultipliers[$validated['intensity']]
  * $typeMultipliers[$validated['activity_type']];

```

33. Ábra: Pontszámítás egy aktivitásért

Összességében az aktivitáskezelés funkció egy átlátható és könnyen használható megoldást nyújt a napi mozgás rögzítésére és követésére. A felhasználó egyszerűen viheti fel az adatait, miközben a rendszer a háttérben gondoskodik azok ellenőrzéséről, feldolgozásáról és tárolásáról.

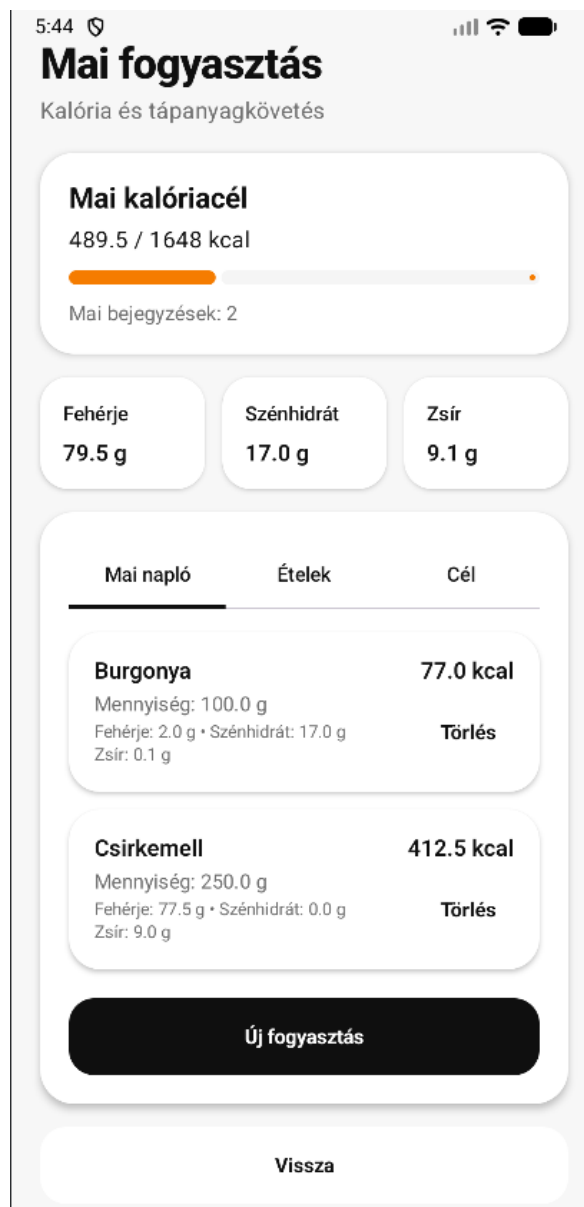
4.5. Táplálkozás kezelése

Az alkalmazás másik központi funkciója a táplálkozás kezelése, amely lehetővé teszi a felhasználó számára a napi étkezések rögzítését és a bevitt tápanyagok nyomon követését. Ez a funkció szorosan kapcsolódik a felhasználó céljaihoz, mivel a napi kalóriabevitel és makrotápanyagok fontos szerepet játszanak a fejlődésben.

A képernyő felső részén a napi kalóriacél jelenik meg, itt is egy 'progress bar' segítségével mutatja, hogy a felhasználó mennyit fogyasztott az adott napi célhoz képest. Emellett megjelenik a napi bejegyzések száma is (34. ábra).

A középső részben a makrotápanyagok (fehérje, szénhidrát, zsír) külön kártyákon jelennek meg, így a felhasználó gyorsan áttekintheti a napi bevitel összetételét (34. ábra).

Az alsó szakasz több fülre van bontva, ahol a felhasználó megtekintheti a napi naplót, az elérhető ételeket, valamint a célok beállítását (34. ábra).



34. Ábra: Fogyasztás képernyő

Az alkalmazás használatához szükséges egy előre feltöltött élelmiszer adatbázis, amelyből a felhasználó kiválaszthatja a fogyasztott ételeket. Ennek biztosítására a backend oldalon egy seedert alkalmaztam (35. és 36. ábra), amely az alapvető élelmiszereket automatikusan betölti az adatbázisba.

```
public function run(): void
{
    $foods = [
        ['name' => 'Alma', 'calories_per_100g' => 52, 'protein_per_100g' => 0.3, 'carbs_per_100g' => 14, 'fat_per_100g' => 0.2],
        ['name' => 'Banán', 'calories_per_100g' => 89, 'protein_per_100g' => 1.1, 'carbs_per_100g' => 23, 'fat_per_100g' => 0.3],
        ['name' => 'Narancs', 'calories_per_100g' => 47, 'protein_per_100g' => 0.9, 'carbs_per_100g' => 12, 'fat_per_100g' => 0.1],
        ['name' => 'Eper', 'calories_per_100g' => 32, 'protein_per_100g' => 0.7, 'carbs_per_100g' => 7.7, 'fat_per_100g' => 0.3],
        ['name' => 'Szőlő', 'calories_per_100g' => 69, 'protein_per_100g' => 0.7, 'carbs_per_100g' => 18, 'fat_per_100g' => 0.2],
    ]
}
```

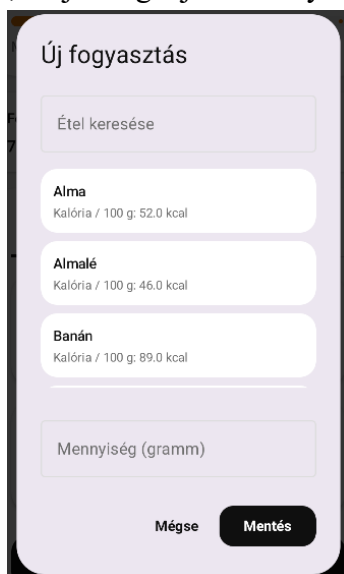
35. Ábra: Ételek tömb egyes elemei és makrói

```
foreach ($foods as $food) {
    Food::updateOrCreate(
        [
            'name' => $food['name'],
            'is_custom' => false,
            'user_id' => null,
        ],
        [
            'calories_per_100g' => $food['calories_per_100g'],
            'protein_per_100g' => $food['protein_per_100g'],
            'carbs_per_100g' => $food['carbs_per_100g'],
            'fat_per_100g' => $food['fat_per_100g'],
        ]
    );
}
```

36. Ábra: Ételek létrehozása

A seeder használatával biztosítható, hogy az alkalmazás már az első indításkor tartalmazzon alapvető élelmiszereket, így a felhasználónak nem szükséges minden adatot manuálisan rögzítenie.

A felhasználó minden étkezést az „Új fogyasztás” gomb segítségével tud rögzíteni. Ilyenkor kiválaszt egy ételt az adatbázisból, majd megadja a mennyiséget grammban (37. ábra).



37. Ábra: Fogyasztás rögzítéséhez felugró ablak

A rendszer ezután automatikusan kiszámolja, hogy a megadott mennyiség alapján mennyi kalóriát és tápanyagot vitt be a felhasználó. Ez a számítás a backend oldalon történik az adott étel 100 grammra vonatkozó értékei alapján (38. ábra).

```
$multiplier = $validated['quantity_grams'] / 100;

$calories = round( num: $food->calories_per_100g * $multiplier, precision: 1);
$protein = round( num: $food->protein_per_100g * $multiplier, precision: 1);
$carbs = round( num: $food->carbs_per_100g * $multiplier, precision: 1);
$fat = round( num: $food->fat_per_100g * $multiplier, precision: 1);
```

38. Ábra: Feljegyzett étkezés makróinak kiszámítása

A számítás után a backend létrehozza az új fogyasztási bejegyzést, amely az aktuális felhasználóhoz kerül mentésre. Ebben a rekordban már a kiszámolt kalória- és tápanyagértékek is szerepelnek, így később ezek közvetlenül felhasználhatók a megjelenítés során (39. ábra).

```
$foodLog = FoodLog::create([
  'user_id' => $user->id,
  'food_id' => $food->id,
  'quantity_grams' => $validated['quantity_grams'],
  'calories' => $calories,
  'protein' => $protein,
  'carbs' => $carbs,
  'fat' => $fat,
  'consumed_date' => Carbon::today()->toDateString(),
]);
```

39. Ábra: Fogyasztási bejegyzés létrehozása

Ez biztosítja, hogy minden fogyasztás külön bejegyzésként kerüljön eltárolásra, és egyértelműen az adott felhasználóhoz tartozzon.

A rögzített fogyasztások a „Mai napló” fül alatt jelennek meg, ahol a felhasználó visszanézheti az adott naphoz tartozó bejegyzéseit. Az egyes elemeknél jól látható az étel neve, a mennyiség, valamint a hozzá tartozó tápértékek (40. ábra).

Mai napló	Ételek	Cél
Burgonya Mennyiség: 100.0 g Fehérje: 2.0 g • Szénhidrát: 17.0 g Zsír: 0.1 g		77.0 kcal Törlés
Csirkemell Mennyiség: 250.0 g Fehérje: 77.5 g • Szénhidrát: 0.0 g Zsír: 9.0 g		412.5 kcal Törlés
Új fogyasztás		

40. Ábra: Fogyasztás: Mai napló fül

A táplálkozás képernyőn külön fül szolgál a célok beállítására, ahol a felhasználó kiválaszthatja, milyen irányba szeretne haladni (például fogyás vagy tömegnövelés). A kiválasztott cél határozza meg a napi kalóriacélt (41. ábra).

Mai napló	Ételek	Cél
Becsült fenntartó kalória: 2148 kcal Jelenlegi cél: Mérsékelt deficit - 1648 kcal		
Agresszív deficit Napi cél: 1348 kcal		
Mérsékelt deficit Napi cél: 1648 kcal		
Súlytartás Napi cél: 2148 kcal		
Mérsékelt tömegnövelés Napi cél: 2648 kcal		
Agresszív tömegnövelés Napi cél: 2948 kcal		
Kalóriacél mentése		

41. Ábra: Fogyasztás: Cél fül

A célokhoz tartozó kalóriaértékek nem kézzel megadott fix számok, hanem a felhasználó adatai alapján kerülnek kiszámításra. A rendszer először meghatározza a becsült fenntartó kalóriaszükségletet (42. ábra), majd ehhez képest von le vagy ad hozzá kalóriát a kiválasztott cél szerint (43. ábra).

```
private function calculateMaintenanceCalories(string $gender, int $age, int $height, float $weight): int
{
    if ($gender === 'male') {
        $bmr = (10 * $weight) + (6.25 * $height) - (5 * $age) + 5;
    } else {
        $bmr = (10 * $weight) + (6.25 * $height) - (5 * $age) - 161;
    }

    $maintenance = $bmr * 1.2;

    return max(value: 1200, (int) round($maintenance));
}
```

42. Ábra: Súlytartó kalória kiszámítása a felhasználó adataihoz

```
private function buildGoalOptions(int $maintenanceCalories): array
{
    return [
        [
            'goal_type' => 'aggressive_cut',
            'label' => 'Aggressive Cut',
            'calorie_target' => max(value: 1200, ...values: $maintenanceCalories - 800),
        ],
        [
            'goal_type' => 'modest_cut',
            'label' => 'Modest Cut',
            'calorie_target' => max(value: 1200, ...values: $maintenanceCalories - 500),
        ],
        [
            'goal_type' => 'maintain',
            'label' => 'Maintain Weight',
            'calorie_target' => $maintenanceCalories,
        ],
        [
            'goal_type' => 'modest_bulk',
            'label' => 'Modest Bulk',
            'calorie_target' => $maintenanceCalories + 500,
        ],
        [
            'goal_type' => 'aggressive_bulk',
            'label' => 'Aggressive Bulk',
            'calorie_target' => $maintenanceCalories + 800,
        ],
    ];
}
```

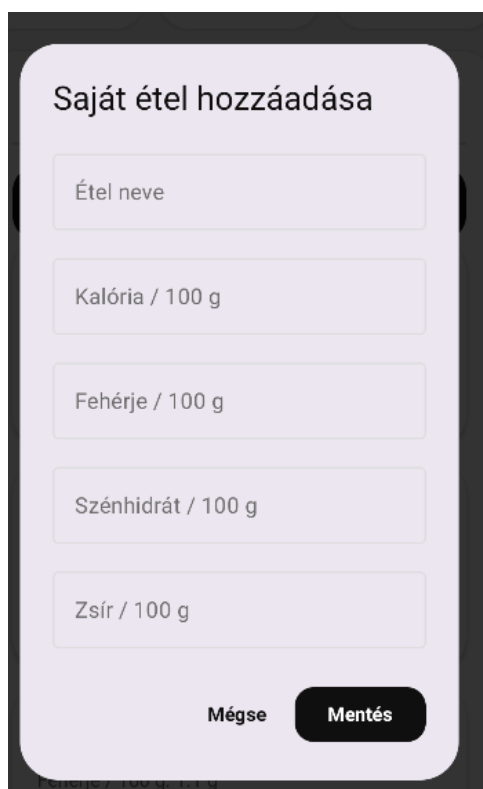
43. Ábra: Az adott felhasználóhoz tartozó kalória opciók

Ennek eredményeként a napi fogyasztás megjelenítése és a 'progress bar' már a kiválasztott célhoz tartozó értékhez viszonyítva működik.

Az alkalmazás lehetőséget ad arra is, hogy a felhasználó saját ételeket vigyen fel (44. ábra). Ez akkor hasznos, ha olyan ételt szeretne rögzíteni, amely nem szerepel az alap adatbázisban.

A felhasználó itt megadja az étel nevét, valamint a 100 grammra vonatkozó tápértékeket.

A backend oldalon ezek az adatok új ételként kerülnek eltárolásra (45. és 46. ábra).



44. Ábra: Saját étel hozzáadása felugró ablak

Ez rugalmasabbá teszi a rendszert, mivel a felhasználó nincs kizárólag az előre feltöltött ételekre utalva.

Összességében a táplálkozás kezelése egy jól átgondolt és könnyen használható rendszert biztosít a napi bevitel követésére. A számítások automatikusan történnek, így a felhasználónak nem kell külön foglalkoznia a tápértékek meghatározásával, miközben folyamatos visszajelzést kap arról, hogyan áll a saját céljaihoz képest.

```
var name by remember { mutableStateOf( value = "") }
var calories by remember { mutableStateOf( value = "") }
var protein by remember { mutableStateOf( value = "") }
var carbs by remember { mutableStateOf( value = "") }
var fat by remember { mutableStateOf( value = "") }
```

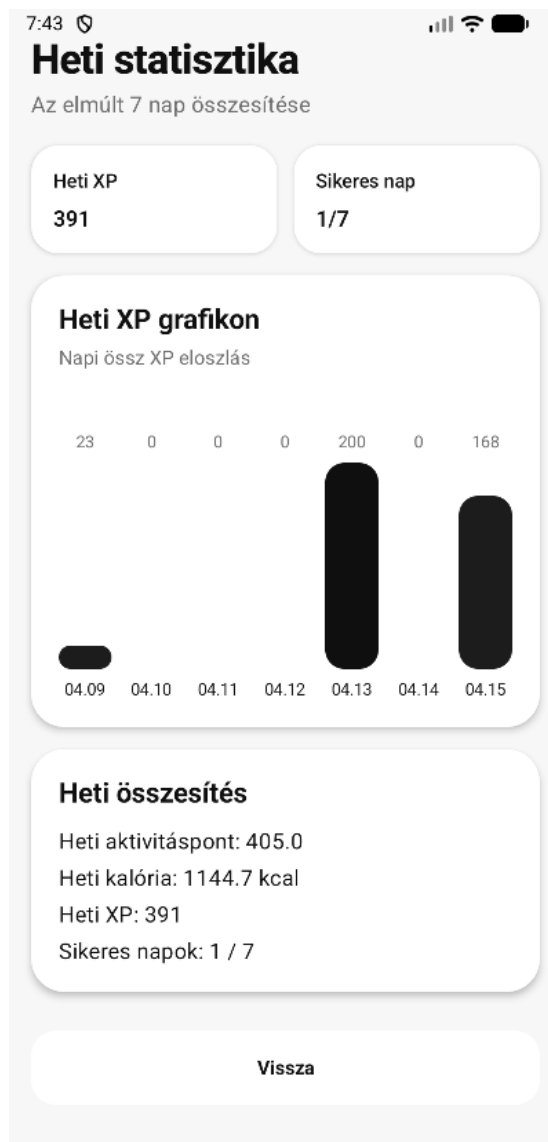
45. Ábra: Egy étel attribútumai

```
$food = Food::create([
    'name' => $validated['name'],
    'calories_per_100g' => $validated['calories_per_100g'],
    'protein_per_100g' => $validated['protein_per_100g'],
    'carbs_per_100g' => $validated['carbs_per_100g'],
    'fat_per_100g' => $validated['fat_per_100g'],
    'is_custom' => true,
    'user_id' => $user->id,
]);
```

46. Ábra: Új étel létrehozása

4.6. Heti statisztika

Az alkalmazás egyik fontos kiegészítő funkciója a heti statisztika megjelenítése, amely lehetőséget ad a felhasználó számára, hogy átfogó képet kapjon az elmúlt napok teljesítményéről. Ez nemcsak visszajelzést ad, hanem motivációt is biztosít a további használathoz.



47. Ábra: Heti statisztika képernyő

A képernyő alsó részén összesített adatok jelennek meg, például a heti összpontszám és a sikeres napok száma. Ezek gyors áttekintést adnak arról, hogy a felhasználó mennyire volt aktív az adott időszakban. Illetve a héten elfogyasztott kalóriamennyiség is látható, ami különösen hasznos lehet a felhasználó számára a céljai követésében (47. ábra).

A statisztika megjelenítéséhez szükséges adatok a backendről kerülnek lekérésre. A frontend egy API hívást indít, amely visszaadja az elmúlt 7 napra vonatkozó adatokat (48. ábra).

```
fun loadStats() {
    loading = true
    errorText = null

    ApiClient.create(context).getWeeklyStats()
        .enqueue( callback = object : Callback<WeeklyStatsResponse> {
            override fun onResponse(
                call: Call<WeeklyStatsResponse>,
                response: Response<WeeklyStatsResponse>
            ) {
                loading = false
                if (response.isSuccessful && response.body() != null) {
                    stats = response.body()
                } else {
                    errorText = "A heti statisztika betöltése sikertelen"
                }
            }
        })
}
```

48. Ábra: API hívás a heti statisztikához

A backend oldalon a rendszer napokra bontva állítja elő az elmúlt hét statisztikáit. Ennek során minden naphoz külön kiszámításra kerül az aktivitásból származó pontérték, az elfogyasztott kalória mennyisége, valamint a hozzá tartozó napi XP adat (49. ábra).

```

for ($i = 6; $i >= 0; $i--) {
    $date = Carbon::today()->subDays($i)->toDateString();

    $activityPoints = round(
        (float) ActivityLog::where('user_id', $user->id)
            ->where('activity_date', $date)
            ->sum('points'),
        precision: 1
    );

    $calories = round(
        (float) FoodLog::where('user_id', $user->id)
            ->where('consumed_date', $date)
            ->sum('calories'),
        precision: 1
    );

    $dailyXp = DailyXpLog::where('user_id', $user->id)
        ->where('xp_date', $date)
        ->first();
}

```

49. Ábra: Napi bontásban számolt statisztika

Ez a megoldás lehetővé teszi, hogy a heti statisztika ne csak összesített formában jelenjen meg, hanem napi bontásban is, ami a grafikon megjelenítéséhez is szükséges.

A backend által visszaadott napi adatok a frontend oldalon grafikus formában kerülnek megjelenítésre. A grafikon megjelenítése során a rendszer minden naphoz kiszámít egy arányt a maximális heti értékhez viszonyítva (50. ábra). Ez az arány határozza meg az adott oszlop magasságát, így a különböző napok teljesítménye vizuálisan is jól összehasonlítható (51. ábra).

```
days.forEach { day ->
    val ratio = day.xp_total.toFloat() / maxXp.toFloat()
```

50. Ábra: Arány számítás a grafikonhoz

```
Box(
    modifier = Modifier
        .fillMaxWidth()
        .height( height = (150f * ratio).dp)
        .background(
            color = if (day.successful) {
                MaterialTheme.colorScheme.primary
            } else {
                MaterialTheme.colorScheme.secondary
            },
            shape = RoundedCornerShape( size = 14.dp)
        )
)
```

51. Ábra: Oszlop megjelenítése

A feldolgozott adatok egy strukturált válasz formájában kerülnek visszaküldésre a frontend számára, amely tartalmazza a napi bontású adatokat, valamint az összesített értékeket is (52. ábra).

```
return response()->json([
    'days' => $days,
    'weekly_activity_points' => round($weeklyActivityPoints, precision: 1),
    'weekly_calories' => round($weeklyCalories, precision: 1),
    'weekly_xp' => $weeklyXp,
    'successful_days' => $successfulDays,
]);
```

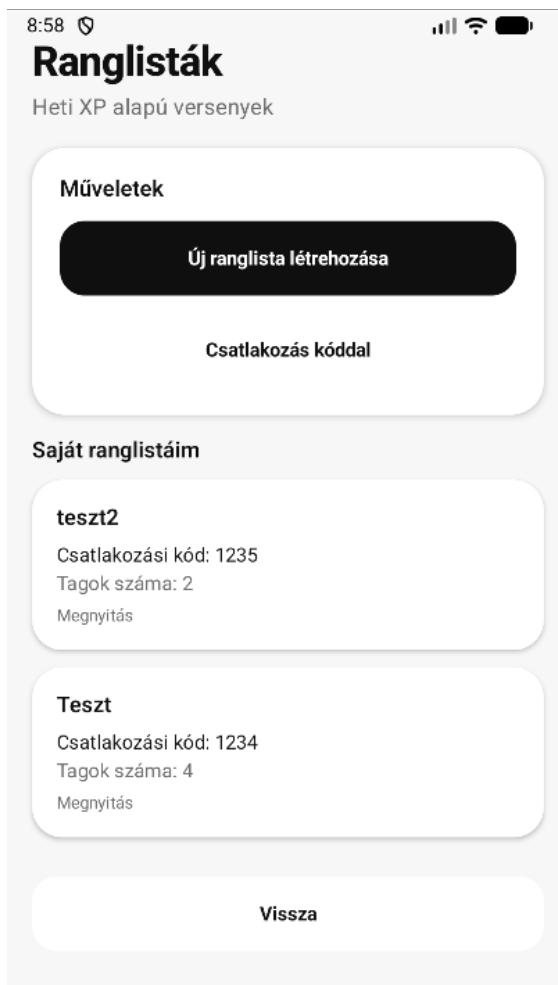
52. Ábra: JSON válasz a felhasználó adatairól

Összességében a heti statisztika funkció lehetőséget biztosít arra, hogy a felhasználó ne csak napi szinten, hanem hosszabb távon is kövesse a teljesítményét. A grafikus megjelenítéssel a felhasználó könnyen átláthatja a saját fejlődését az idő múlásával.

4.7. Ranglista rendszer

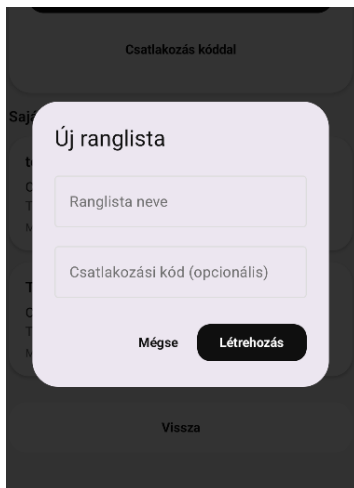
A ranglista rendszer célja, hogy a felhasználók ne csak saját teljesítményüket követhessék, hanem mások eredményeivel is össze tudják hasonlítani azt. Ez közösségi és

motivációs szerepet is betölt az alkalmazásban, mivel a heti XP alapján kialakuló sorrend versenyhelyzetet teremt a résztvevők között.

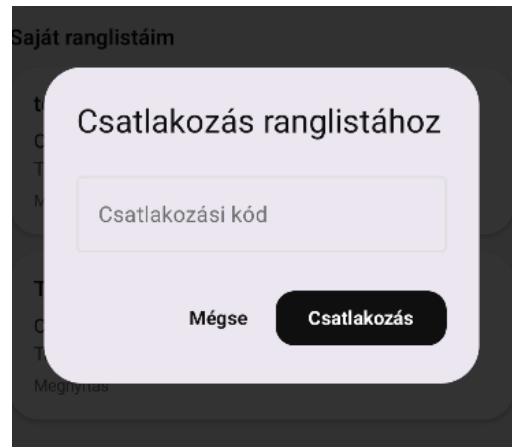


53. Ábra: Ranglista képernyő

A ranglista képernyőn a felhasználó megtekintheti azokat a ranglistákat, amelyeknek tagja, új ranglistát hozhat létre, illetve csatlakozhat egy már létezőhöz egy egyedi kód segítségével (53., 54. és 55. ábra).



54. Ábra: Új ranglista felugró ablak



55. Ábra: Csatlakozás ranglistához felugró ablak

A frontend oldalon a létrehozás egy szokásos API hívással történik. A backend oldalon pedig a rendszer először validálja a beérkező adatokat, majd létrehozza a ranglistát. A validáció után a ranglista rekord létrejön, majd a rendszer automatikusan hozzáadja a létrehozó felhasználót a tagok közé is (56. ábra). Ez a megoldás biztosítja, hogy a létrehozó felhasználó rögtön a ranglista tagja legyen, és ne kelljen külön csatlakoznia.

```
$validated = $request->validate([
    'name' => 'required|string|max:255',
    'join_code' => 'nullable|string|min:4|max:20|unique:leaderboards,join_code',
]);

$joinCode = $validated['join_code'] ?? $this->generateJoinCode();

$leaderboard = Leaderboard::create([
    'name' => $validated['name'],
    'join_code' => strtoupper($joinCode),
    'created_by' => $user->id,
]);

LeaderboardMember::create([
    'leaderboard_id' => $leaderboard->id,
    'user_id' => $user->id,
]);
```

56. Ábra: Ranglista validálása majd létrehozása, és a felhasználó hozzáadása

A ranglista részletes nézetében a rendszer a tagokat a heti XP alapján rendezi sorba. Ehhez először meghatározza az aktuális hét időintervallumát, majd összegyűjti a ranglista tagjainak XP értékeit (57. ábra).

```

$weeklyXpMap = DailyXpLog::whereIn('user_id', $memberIds)
    ->whereBetween('xp_date', [$weekStart, $weekEnd])
    ->get()
    ->groupBy('user_id')
    ->map(function ($logs) {
        return (int) $logs->sum('total_xp');
    });

```

57. Ábra: A ranglista tagjainak és időintervallumának meghatározása

Ezután a rendszer minden felhasználóhoz hozzárendeli a heti XP értéket, majd csökkenő sorrendbe rendezi őket (58. ábra).

```

$rankedMembers = $leaderboard->members
    ->map(function ($member) use ($weeklyXpMap) {
        return [
            'user_id' => $member->user->id,
            'name' => $member->user->name,
            'weekly_xp' => $weeklyXpMap[$member->user->id] ?? 0,
        ];
    })
    ->sortByDesc('weekly_xp')
    ->values();

```

58. Ábra: Csökkenő sorrendbe rendezés XP szerint

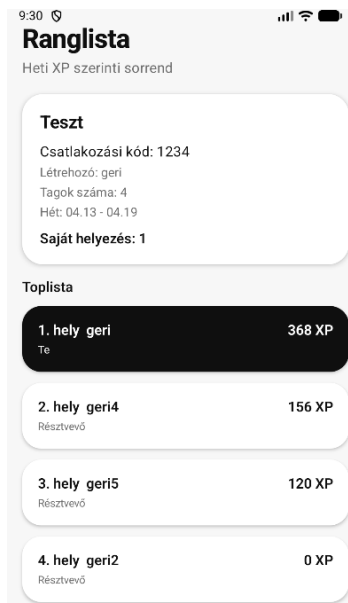
A rendezés után a rendszer meghatározza a helyezéseket is, valamint megjelöli az aktuális felhasználót (59. ábra).

```

$rankedMembers = $rankedMembers->map(function ($member, $index) use ($user) {
    return [
        'rank' => $index + 1,
        'user_id' => $member['user_id'],
        'name' => $member['name'],
        'weekly_xp' => $member['weekly_xp'],
        'is_me' => $member['user_id'] === $user->id,
    ];
})->values();

```

59. Ábra: Helyezések meghatározása



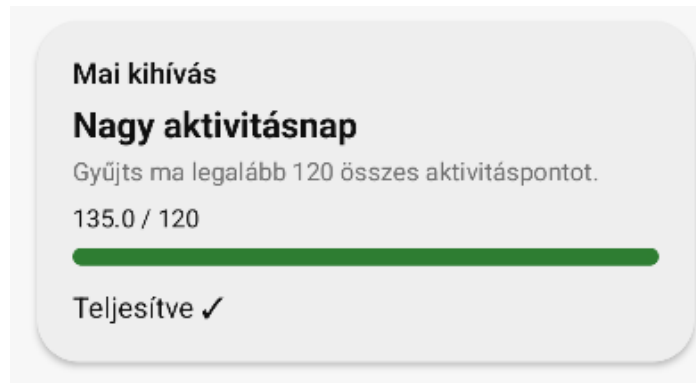
60. Ábra: Egy adott ranglista képernyője

A frontend oldalon a felhasználók sorrendje már a backend által rendezett formában érkezik meg, így a megjelenítés egyszerű listaként történik. A saját felhasználó kiemelésre kerül, ami segíti a gyors áttekintést (60. ábra).

Összességében a ranglista rendszer egy egyszerű, de hatékony módja annak, hogy a felhasználók összehasonlítsák egymás teljesítményét. A heti XP alapú rangsorolás közvetlen visszajelzést ad, és versenyhelyzetet teremt, amely ösztönzőleg hathat a rendszeres használatra.

4.8. Napi kihívások

Az alkalmazás utolsó gamifikációs eleme a napi kihívások rendszere, amely extra motivációt biztosít a felhasználó számára a rendszeres használathoz. A kihívások célja, hogy kisebb, könnyen teljesíthető feladatokon keresztül ösztönözzék a felhasználót az aktivitás és a táplálkozás folyamatos követésére.



61. Ábra: Kihívás kártya a főképernyőn

A főképernyőn a felhasználó mindig egy aktuális napi kihívást lát, amely tartalmazza a feladat leírását, valamint a teljesítéshez szükséges feltételeket. Emellett a rendszer jelzi az aktuális állapotot is (61. ábra).

A napi kihívások nem dinamikusan generált feladatok, hanem egy előre összeállított listából kerülnek kiválasztásra. A rendszer összesen 30 különböző kihívást tartalmaz, amelyeket a backend oldalon egy seeder segítségével töltöttem fel az adatbázisba (62. és 63. ábra).

```
class ChallengeSeeder extends Seeder
{
    // Abari Gergő
    public function run(): void
    {
        $challenges = [
            [
                'title' => 'Sétaindító',
                'description' => 'Teljesíts ma legalább 20 perc walking aktivitást.',
                'challenge_type' => 'activity_duration',
                'category' => 'cardio',
                'activity_type' => 'walking',
                'target_value' => 20,
            ],
        ],
```

62. Ábra: Egy kihívás attribútumai

```
foreach ($challenges as $challenge) {
    Challenge::create([
        ...$challenge,
        'reward_xp' => 50,
        'is_active' => true,
    ]);
}
```

63. Ábra: Kihívás létrehozása

A kihívás teljesítése nem külön művelethez kötött, hanem az alkalmazás más funkcióihoz kapcsolódik. Például amikor a felhasználó aktivitást vagy fogyasztást rögzít, a rendszer automatikusan frissíti az állapotot (64. ábra).

```

private function evaluateChallengeWithProgress(User $user, UserDailyChallenge $dailyChallenge): array
{
    $challenge = $dailyChallenge->challenge;
    $today = Carbon::today()->toDateString();

    $current = 0;
    $target = $challenge->target_value;

    switch ($challenge->challenge_type) {

        case 'activity_duration':
            $current = (int) ActivityLog::where('user_id', $user->id)
                ->where('activity_date', $today)
                ->where('activity_type', $challenge->activity_type)
                ->sum('duration_minutes');
            break;

        case 'activity_points':
            $current = (float) ActivityLog::where('user_id', $user->id)
                ->where('activity_date', $today)
                ->where('category', $challenge->category)
                ->sum('points');
            break;
    }
}

```

64. Ábra: Kihívás frissítése ha a felhasználó rögzít valamit

A rendszer a kihívás típusától függően különböző adatokat vizsgál. Egyes kihívások az aktivitások időtartamára, mások az összegyűjtött pontokra vagy a bevitt tápanyagokra épülnek.

A kiszámított aktuális érték alapján a rendszer eldönti, hogy a kihívás teljesült-e (65. ábra). Amennyiben az aktuális érték eléri vagy meghaladja a célértéket, teljesítettnek minősül. Sikeres teljesítés esetén a felhasználó jutalmat kap, amely XP formájában kerül jóváírásra.

```

$isCompleted = $current >= $target;

return [
    'is_completed' => $isCompleted,
    'current_value' => round($current, precision: 1),
    'target_value' => $target,
    'progress_ratio' => $target > 0 ? min( value: $current / $target, ...values: 1) : 0,
];

```

65. Ábra: Kihívás teljesülésének ellenőrzése

A rendszer biztosítja, hogy a jutalom csak egyszer kerüljön jóváírásra, így elkerülhető a többszöri XP szerzés ugyanazon kihívás teljesítésével.

Az alkalmazás minden nap egy véletlenszerű kihívást rendel a felhasználóhoz, figyelembe véve, hogy a közelmúltban teljesített kihívások ne ismétlődjenek túl gyakran (66. ábra).

```
$challenge = $query->inRandomOrder()->first();

if (!$challenge) {
    $challenge = Challenge::where('is_active', true)
        ->inRandomOrder()
        ->first();
}

return UserDailyChallenge::create([
    'user_id' => $user->id,
    'challenge_id' => $challenge->id,
    'challenge_date' => $today,
    'is_completed' => false,
    'completed_at' => null,
    'reward_granted' => false,
])->load('challenge');
```

66. Ábra: Véletlenszerű kihívás rendelése a felhasználóhoz

Összességében a napi kihívások rendszere egyszerűen illeszkedik az alkalmazás többi funkciójához, és folyamatos visszajelzést ad a felhasználónak a haladásáról. Az automatikus értékelés és a jutalmazási rendszer hozzájárul a felhasználói motiváció fenntartásához, miközben nem igényel külön beavatkozást a felhasználó részéről.

4.9. Továbbfejlesztési lehetőségek

Bár az alkalmazás jelenlegi formájában is alkalmas a napi aktivitás, táplálkozás és a kapcsolódó gamifikációs elemek kezelésére, több olyan irány is van, amelyben a rendszer a jövőben tovább bővíthető.

Az egyik kézenfekvő fejlesztési lehetőség az értesítési rendszer bevezetése lenne. Push értesítések segítségével a felhasználó emlékeztetőt kaphatna a napi kihívásról, a fogyasztás vagy aktivitás rögzítéséről, illetve a heti statisztika elérhetőségéről. Ez tovább növelhetné az alkalmazás használatának rendszerességét.

További bővítési lehetőséget jelentene a napi kihívások rendszerének fejlesztése. A jelenlegi, előre definiált kihívások mellett később személyre szabott, a felhasználó korábbi

viselkedéséhez igazodó feladatok is megjelenhetnének. Ez még inkább egyedivé és motiválóvá tehetné a rendszert.

Hasznos fejlesztés lenne az adatok részletesebb elemzése is. A heti statisztikák mellett havi vagy akár hosszabb távú grafikonok is megjelenhetnének, amelyek segítségével a felhasználó jobban átláthatná a fejlődését és szokásainak változását.

A táplálkozási modul esetében szintén több bővítési lehetőség kínálkozik. Ilyen lehet például vonalkód alapú élelmiszer-felismerés, recept alapú ételrögzítés vagy külső élelmiszer-adatbázisok integrálása, amelyek tovább egyszerűsíthetnék a napi használatot.

Végül technikai szempontból is elképzelhetők további fejlesztések. Ilyen lehet például az offline működés támogatása vagy a felhasználói felület további finomítása.

Összességében az alkalmazás jelenlegi formájában egy jól működő alapot biztosít, amelyre a jövőben több irányban is tovább lehet építeni, mind funkcionalitás, mind felhasználói élmény szempontjából.

Irodalomjegyzék

- [1] P. T. Katzmarzyk, C. Friedenreich, E. J. Shiroma, és I.-M. Lee, „Physical inactivity and non-communicable disease burden in low-income, middle-income and high-income countries”, *Br. J. Sports Med.*, köt. 56, sz. 2, o. 101–106, jan. 2022, doi: 10.1136/bjsports-2020-103640.
- [2] S. J. Hardcastle, J. Hancox, A. Hattar, C. Maxwell-Smith, C. Thøgersen-Ntoumani, és M. S. Hagger, „Motivating the unmotivated: how can health behavior be changed in those unwilling to change?”, *Front. Psychol.*, köt. 6, jún. 2015, doi: 10.3389/fpsyg.2015.00835.
- [3] S. Deterding, D. Dixon, R. Khaled, és L. Nacke, „From Game Design Elements to Gamefulness: Defining »Gamification«”, in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments, 2011, o. 9–15. doi: 10.1145/2181037.2181040.
- [4] E. A. Edwards és mtsai., „Gamification for health promotion: systematic review of behaviour change techniques in smartphone apps”, *BMJ Open*, köt. 6, sz. 10, o. e012447, okt. 2016, doi: 10.1136/bmjopen-2016-012447.
- [5] S. Deterding, D. Dixon, R. Khaled, és L. Nacke, „From game design elements to gamefulness: defining »gamification«”, in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, Tampere Finland: ACM, szept. 2011, o. 9–15. doi: 10.1145/2181037.2181040.
- [6] S. Hallifax, M. Altmeyer, K. Kölln, M. Rauschenberger, és L. E. Nacke, „From Points to Progression: A Scoping Review of Game Elements in Gamification Research with a Content Analysis of 280 Research Papers”, *Proc. ACM Hum.-Comput. Interact.*, köt. 7, sz. CHI PLAY, o. 748–768, szept. 2023, doi: 10.1145/3611048.
- [7] J. Koivisto és J. Hamari, „The rise of motivational information systems: A review of gamification research”, *Int. J. Inf. Manag.*, köt. 45, sz. July 2018, o. 191–210, 2019, doi: 10.1016/j.ijinfomgt.2018.10.013.
- [8] M. Sailer és L. Homner, „The Gamification of Learning: a Meta-analysis”, *Educ. Psychol. Rev.*, köt. 32, sz. 1, o. 77–112, márc. 2020, doi: 10.1007/s10648-019-09498-w.
- [9] D. Johnson, S. Deterding, K.-A. A. Kuhn, A. Staneva, S. Stoyanov, és L. Hides, „Gamification for health and wellbeing: A systematic review of the literature”, *Internet Interv.*, köt. 6, o. 89–106, 2016, doi: 10.1016/j.invent.2016.10.002.
- [10] C. Almeida, M. Kalinowski, A. Uchôa, és B. Feijó, „Negative effects of gamification in education software: Systematic mapping and practitioner perceptions”, *Inf. Softw. Technol.*, köt. 156, o. 107142, 2023, doi: 10.1016/j.infsof.2022.107142.
- [11] D. Van Zandvoort, M. Vredenburg, és M. Bentvelzen, „Unhealthy Comparisons to Promote Healthy Behavior? Exploring the Impact of Social Comparison Strategies in Personal Informatics.”, in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, Yokohama Japan: ACM, ápr. 2025, o. 1–21. doi: 10.1145/3706598.3713737.
- [12] „What is PHP?” [Online]. Elérhető: <https://php-legacy-docs.zend.com/manual/php5/en/intro-what-is>
- [13] „Laravel Documentation”. [Online]. Elérhető: <https://laravel.com/docs/13.x>

- [14] M. Jones és D. Hardt, „The OAuth 2.0 Authorization Framework: Bearer Token Usage”, RFC Editor, RFC6750, okt. 2012. doi: 10.17487/rfc6750.
- [15] „Laravel Documentation Middleware”. [Online]. Elérhető: <https://laravel.com/docs/13.x/middleware>
- [16] „Laravel Documentation Hashing”. [Online]. Elérhető: <https://laravel.com/docs/13.x/hashing>
- [17] „MySQL Reference Manual Acid”. [Online]. Elérhető: <https://dev.mysql.com/doc/refman/8.4/en/mysql-acid.html>
- [18] R. Wodyk és M. Skublewska-Paszkowska, „Performance comparison of relational databases SQL Server, MySQL and PostgreSQL using a web application and the Laravel framework”, *J. Comput. Sci. Inst.*, köt. 17, o. 358–364, dec. 2020, doi: 10.35784/jcsi.2279.
- [19] „Laravel Documentations Migrations”. [Online]. Elérhető: <https://laravel.com/docs/13.x/migrations>
- [20] „Laravel Documentation Eloquent”. [Online]. Elérhető: <https://laravel.com/docs/13.x/eloquent>
- [21] „Android Studio Documentation Intro”. [Online]. Elérhető: <https://developer.android.com/studio/intro>
- [22] „Android’s Kotlin-first approach”. [Online]. Elérhető: <https://developer.android.com/kotlin/first>
- [23] T. Trinadh, „Android UI: Jetpack Compose vs. Views - The Definitive Shift (And What It Means For You)”. [Online]. Elérhető: <https://dev.to/trinadhthatakula/android-ui-jetpack-compose-vs-views-the-definitive-shift-and-what-it-means-for-you-3gi0>
- [24] „OkHttp”. [Online]. Elérhető: <https://square.github.io/okhttp/>
- [25] „Retrofit”. [Online]. Elérhető: <https://square.github.io/retrofit/>

Nyilatkozat mesterséges intelligencia eszközök használatáról

Ezúton nyilatkozom, hogy a szakdolgozat elkészítése során az alábbi mesterséges intelligencia alapú eszköz(ök)et használtam / nem használtam.

1. MI-eszköz használata

X Nem használtam mesterséges intelligencia alapú eszközt a szakdolgozat készítése során.

☐ Használtam mesterséges intelligencia alapú eszközt a szakdolgozat készítése során az alábbiak szerint:

MI eszköz neve és verziója	Pontosan mire használtam
----------------------------	--------------------------

.....

.....

.....

.....

.....

.....

(A sorok száma bővíthető. Példák a használat részletezésére: nyelvi javítás, stilisztikai finomítás, helyesírási ellenőrzés, szöveg rövidítése vagy átfogalmazása, programkód magyarázata vagy hibakeresése, irodalomkeresési ötletek gyűjtése, ábrák, táblázatok vagy prezentációs elemek előkészítése, adatfeldolgozási vagy modellezési ötletek ellenőrzése. - Ez a zárójeles rész a szakdolgozatból törlendő.)

2. Nyilatkozat az önálló munkáról

Kijelentem, hogy a szakdolgozat szakmai tartalmáért teljes felelősséget vállalok. Tudomásul veszem, hogy a mesterséges intelligencia alapú eszközök használata nem helyettesíti az önálló munkát, ezért minden, a dolgozatban szereplő állítást, eredményt, értelmezést, következtetést és hivatkozást ellenőriztem.

Kijelentem továbbá, hogy:

- a dolgozatban szereplő végleges tartalmat én állítottam össze,
- az MI által generált vagy módosított tartalmakat átnéztem, ellenőriztem és szükség esetén javítottam,
- a dolgozatban bemutatott saját eredmények, értelmezések és következtetések az én felelősségem alá tartoznak,
- jelen nyilatkozatban az MI-használatot valósághűen és teljes körűen feltüntettem.